

MASTER PROJECT DOCUMENT

National 3D ULPIN & Volumetric Cadastre Platform

Smart India Hackathon (SIH) — PS 26011 | Ministry of Rural Development —
Department of Land Resources (DoLR)

Document Version: 1.0

Date: September 2026

Prepared by: Technical Architecture & Documentation Team

Confidentiality: Public — Suitable for Hackathon, Investor, and Academic Evaluation

INFORMATION GAPS AND ASSUMPTIONS

Before proceeding, the following information gaps are identified. Where data is unavailable, clearly labeled assumptions have been made.

#	Missing Information	Assumption Made	Impact
1	Production deployment infrastructure details	Assumed local development deployment; cloud recommendations are illustrative	Deployment & Scalability sections
2	Exact team composition and budget	Assumed typical SIH 6-member student team	Role & financial sections
3	Real-world adoption data / user metrics	No metrics claimed; illustrative estimates are labeled	Impact & KPI sections
4	Government API access / Aadhaar integration specifics	Marked as future scope; CERSAI integration is mocked	Authentication & API sections
5	Load testing / benchmark results	No benchmarks conducted yet; estimates labeled	Performance section
6	Legal counsel review of compliance claims	ISO 19152 alignment is structural, not legally certified	Compliance section
7	Drone photogrammetry field data	Point cloud validation pipeline is built but tested with synthetic/optional data	AI/ML section
8	Financial projections / revenue model	Not applicable for SIH context; illustrative notes provided	Business section

[NOTE] Every section below is **specifically connected to the actual codebase and project files**. Generic filler has been avoided. Assumptions are clearly marked with **[ASSUMPTION]** tags.

TABLE OF CONTENTS

- [Section 1 — Project Identity](#)
- [Section 2 — Context and Background](#)
- [Section 3 — Problem Statement](#)
- [Section 4 — Project Purpose](#)
- [Section 5 — Target Users and Stakeholders](#)
- [Section 6 — Existing Solutions and Gap Analysis](#)
- [Section 7 — Proposed Solution](#)
- [Section 8 — System Architecture](#)
- [Section 9 — Complete Technology Stack](#)
- [Section 10 — Technology Deep Dive](#)
- [Section 11 — Feature Breakdown](#)
- [Section 12 — Complete User Flow](#)
- [Section 13 — Data Flow Architecture](#)
- [Section 14 — Database Design](#)
- [Section 15 — API Architecture](#)
- [Section 16 — AI / Machine Learning Architecture](#)
- [Section 17 — Algorithms and Core Logic](#)

- 18. [Section 18 — Security Architecture](#)
- 19. [Section 19 — Scalability Architecture](#)
- 20. [Section 20 — Performance Analysis](#)
- 21. [Section 21 — Impact Analysis](#)
- 22. [Section 22 — Success Metrics and KPIs](#)
- 23. [Section 23 — Risk Analysis](#)
- 24. [Section 24 — Limitations](#)
- 25. [Section 25 — Future Roadmap](#)
- 26. [Section 26 — Implementation Roadmap](#)
- 27. [Section 27 — Testing Strategy](#)
- 28. [Section 28 — Deployment Architecture](#)
- 29. [Section 29 — Project Impact Visualization](#)
- 30. [Section 30 — System Diagrams](#)
- 31. [Section 31 — Project Story](#)
- 32. [Section 32 — Pitch Version](#)
- 33. [Section 33 — FAQ](#)
- 34. [Section 34 — Master Technology Interaction Map](#)
- 35. [Section 35 — Final Project Summary](#)

SECTION 1 — PROJECT IDENTITY

1. Project Name

National 3D ULPIN & Volumetric Cadastre Platform

2. Project Tagline

"Every room. Every floor. Every right. One 3D identity."

3. One-line Project Description

An enterprise ISO 19152-compliant system that extends India's 2D ULPIN (Unique Land Parcel Identification Number / Bhu-Aadhaar) into the vertical dimension, generating unique, validated 3D spatial identifiers for every room, corridor, and utility shaft in multi-storey buildings.

4. Elevator Pitch

India's existing land records identify surface plots with 2D identifiers. But in a 10-storey apartment building, 50 families share the exact same latitude-longitude coordinates. Who owns what? Our platform takes architectural CAD floor plans, extracts room boundaries using computer vision, extrudes them vertically with measured heights, assigns each unit a standardized 3D ULPIN, validates the entire geometry for collisions, and links it to ISO 19152-compliant ownership rights — creating India's first volumetric cadastre that can tell you not just *which plot*, but *which exact 3D airspace volume* belongs to whom.

5. Detailed Project Overview

The **National 3D ULPIN & Volumetric Cadastre Platform** is a working technical prototype developed for Smart India Hackathon Problem Statement 26011, issued by the Department of Land Resources (DoLR), Ministry of Rural Development, Government of India.

The system addresses a fundamental limitation of India's existing land administration: the inability of 2D cadastral records to represent, identify, and administer property rights in multi-storey structures. Traditional cadastral systems represent land as 2D polygons on a surface map. When multiple apartments, commercial units, corridors, utility shafts, and parking areas are stacked vertically above the same ground parcel, the 2D system cannot distinguish between them.

This platform provides:

- **Automated extraction** of room boundaries from architectural CAD PDFs (vector) and scanned floor plan images (raster + OCR)
- **3D volumetric extrusion** of extracted 2D rooms across multiple floors with precise elevation data
- **WGS84 GPS georeferencing** of every 3D unit to real-world coordinates
- **Standardized 3D ULPIN generation** encoding building, floor, unit, and zoning type
- **3D topology validation** using computational geometry to detect spatial collisions
- **ISO 19152 LADM v2 Rights-Restrictions-Responsibilities (RRR)** registry for strata titles, mortgage liens, and conveyance mutations
- **Interactive WebGL 3D Digital Twin** for visual inspection of the volumetric cadastre

- **Drone photogrammetry ML validation** pipeline for cross-validating building heights against point cloud data
- **RESTful API backend** with 8 modular routers serving the entire cadastral ecosystem

The prototype is demonstrated on a real-world dataset: the 10-storey Hostel Block A at the Indian Institute of Information Technology, Nagpur campus, producing **700 verified 3D volumetric parcels** (560 residential + 140 common/utility) with real surveyed architectural data from Government of Maharashtra's Bhunaksha/Mahabhulekh records (Survey No. 140/1, Khata No. 341, Village Waranga, Nagpur Rural).

6. Project Category

Software | Smart Automation | GovTech / Land Administration

7. Domain / Industry

- Government Land Administration (Cadastral Systems)
- Geospatial Information Systems (GIS)
- PropTech / Real Estate Technology
- Urban Planning & Smart Cities

8. Target Users

User Type	Description
Land Revenue Officers	State officials administering property records and mutations
Sub-Registrars	Officers registering property deeds and transfers
Surveyors / Architects	Professionals digitizing building floor plans
Property Owners	Citizens verifying their 3D property boundaries and titles
Banks / Financial Institutions	Entities stamping mortgage liens against specific 3D units
Municipal Corporations	Urban local bodies managing building approvals and zoning
RERA Authorities	Real Estate Regulatory Authority verifying carpet area compliance
Policy Makers	Government decision-makers shaping land reform policy

9. Stakeholders

- Ministry of Rural Development — Department of Land Resources (DoLR)
- National Informatics Centre (NIC) / DILRMP Programme
- State Revenue Departments (Maharashtra, etc.)
- Smart India Hackathon Organizers & Evaluators
- IIIT Nagpur (Prototype host institution)

10. Geographic Scope

- **Prototype:** IIIT Nagpur Campus, Village Waranga, Taluka Nagpur Rural, District Nagpur, Maharashtra
- **Design Target:** National deployment across all Indian states (framework is state-agnostic)

11. Current Development Stage

Working Prototype (MVP) — Demonstrated on verified real-world data, ready for hackathon evaluation and field pilot scaling.

Three-Level Explanation

LEVEL 1 — For a Non-Technical Person

Imagine you live in a 10-floor apartment building. The government has a "property ID" for the land your building sits on — but it's the same ID for every single flat in the building. If you want to prove you own Flat 502, the current system only says "you own a share of this plot of land." It can't point to the exact 3D space that is your flat.

Our project fixes this. We take the architectural drawings of a building, use a computer to read and understand every room, corridor, staircase, and lift shaft, then stack them floor by floor into a 3D model. Every single room gets its own unique "3D address" — like an Aadhaar card, but for a specific room in 3D space. We can then say: "This room is 5.0m × 7.0m, sits between 6.8m and 9.7m elevation, is at latitude 20.9495556° N, and is legally held by Person X." We can also check if any two rooms accidentally overlap in 3D, prevent overcrowding, and block illegal sales of mortgaged units.

LEVEL 2 — For a College Student / Project Evaluator

This project extends India's 2D Unique Land Parcel Identification Number (ULPIN / Bhu-Aadhaar) into 3D. The existing DILRMP (Digital India Land Records Modernisation Programme) assigns 14-digit identifiers to surface plots, but has no mechanism for vertical property — apartments, offices, underground parking, etc.

Our system implements a complete geospatial pipeline:

1. **Input:** Vector CAD PDFs or scanned floor plan images + building GPS anchor + measured floor heights
2. **Extraction:** PyMuPDF for vector drawings; OpenCV + Tesseract OCR for scanned images
3. **Georeferencing:** Local pixel coordinates → metric Cartesian → WGS84 latitude/longitude via affine transform
4. **3D Stacking:** Each extracted floor is duplicated and extruded vertically at 3.4m pitch intervals
5. **ULPIN Generation:** Composite identifier schema `BUILDING-FLOOR-UNIT-TYPE` (e.g., `HSTL01-F5-501-4S`)
6. **Topology Validation:** Shapely 2.0 performs Z-interval overlap detection + 2D polygon intersection area computation → 0 collisions across 700 units
7. **Rights Registry:** ISO 19152 LADM v2-compliant strata title allotment, conveyance mutation with anti-fraud lien checks, and immutable audit logging
8. **Visualization:** Plotly WebGL 3D mesh rendering with per-floor filtering and hover inspection
9. **ML Validation:** Unsupervised DBSCAN clustering on drone photogrammetry point cloud Z-coordinates to cross-validate stated floor counts

Demonstrated on 700 genuine volumetric parcels across 10 floors of a real hostel building, with data sourced from Government of Maharashtra Bhunaksha/Mahabhulekh.

LEVEL 3 — For a Technical Architect / Experienced Developer

The system is a modular, RESTful 3D cadastral information system built on:

- **Backend:** FastAPI 0.110+ with 8 routers (`institutions, buildings, parcels, rights, encumbrances, topology, analytics, ingestion`) serving JSON over OpenAPI 3.0
- **Database:** SQLite with 7 relational tables (`institutions, buildings, parcels_3d, parties, strata_titles, encumbrances, mutation_audit_log`), foreign key constraints, and indexed spatial queries
- **Extraction Pipeline:** Dual-mode ingestion supporting vector PDF parsing (PyMuPDF `get_drawings()` + text extraction) and raster CV pipeline (OpenCV adaptive thresholding → morphological closing → contour detection with NMS → Tesseract OCR with confidence scoring)
- **Coordinate System:** Local CAD coordinates are scaled via a calibrated factor (0.1362 m/pt for the prototype) with optional horizontal flip, then georeferenced to WGS84 via `METERS_PER_DEG_LAT = 111320.0` and `cos(lat)` correction for longitude
- **3D ULPIN Schema:** Hierarchical composite primary key `{BUILDING_CODE}-{FLOOR}-{UNIT_ID}-{ZONING_TYPE}` ensuring global uniqueness within the estate
- **Topology Engine:** $O(N^2/\text{floor})$ pairwise Shapely `box()` intersection test with Z-interval pre-filtering, 0.05 m² tolerance for boundary adjacency, exempting common halls
- **RRR Engine:** Implements ISO 19152 `LA_BAUnit`, `LA_Party`, `LA_RRR` concepts with statutory capacity enforcement (`4S → max 4, 2S → max 2`), inalienable common property locks, active encumbrance conveyance blocks, and immutable mutation audit trail
- **ML Height Validation:** Point cloud Z-coordinate histogram density analysis → `scipy.signal.find_peaks` for floor slab detection → `sklearn.cluster.DBSCAN` for unsupervised elevation clustering → discrepancy flagging against config
- **Frontend:** Static HTML/CSS/JS with Plotly WebGL 3D mesh rendering, Mahabhulekh-style cascading search UI, and bilingual (English/Hindi) landing pages
- **Data Provenance:** Anchored to verified Government of Maharashtra Bhunaksha record — State Parcel ID `pu-id: 33550994106`, Survey 140/1, Khata 341, Village Waranga

The architecture is designed for generalization: the `BuildingConfig` + `RoomTypeRule` JSON schema plus `RuleEngine` pattern allow any new building to be onboarded by providing a floor plan + configuration file, with zero Python code changes.

SECTION 2 — CONTEXT AND BACKGROUND

Industry Background

India's land administration system is one of the largest and most complex in the world, covering approximately **620 million land parcels** across 28 states and 8 union territories. The **Digital India Land Records Modernisation Programme (DILRMP)**, launched under the Department of Land Resources (DoLR), has been working to digitize these records and assign each surface parcel a unique 14-digit **ULPIN (Unique Land Parcel Identification Number)**, often called "Bhu-Aadhaar."

However, the entire DILRMP framework operates in **two dimensions**. Every identifier, every record, and every spatial reference treats land as a flat surface polygon defined by (latitude, longitude) coordinates.

The Existing Situation

Aspect	Current State
Land Record Format	2D polygon (surface parcel) with alphanumeric identifier
Vertical Representation	None — multi-storey buildings have no floor/unit-level identity
Ownership Granularity	Entire plot or "undivided share" — no individual apartment identity
Spatial Validation	Manual comparison of survey maps; no automated 3D collision detection
Rights Registry	Disconnected from spatial geometry; text-based deed registers
Digital Twin	Non-existent for most Indian buildings
International Compliance	No ISO 19152 LADM alignment in operational cadastre systems

Why This Problem Matters Now

Several converging forces make this problem urgent:

- Rapid Urbanization:** India is projected to have 600+ million urban residents by 2030 (UN Habitat). Multi-storey construction is the dominant mode.
- RERA Enforcement:** The Real Estate (Regulation and Development) Act, 2016 mandates precise carpet area disclosure — but there's no spatial system to verify it.
- Smart Cities Mission:** 100 Smart Cities need digital building-level data for planning, utilities, and emergency services.
- Financial Inclusion:** Banks issuing home loans need to identify the *specific unit* being mortgaged, not just "a share of plot 140/1."
- Land Disputes:** Land litigation accounts for 66% of all civil cases in Indian courts (NITI Aayog). Many disputes involve vertical property boundaries that 2D records cannot resolve.
- International Standards:** India seeks compliance with ISO 19152 LADM (Land Administration Domain Model), which explicitly supports 3D cadastre concepts.

What Happens Today Without This Project?

Current Workflow (Without 3D ULPIN):

- A 10-storey building with 100 apartments is built on Surface Parcel Survey No. 140/1
- The **entire building** gets one 2D ULPIN covering the footprint
- Individual apartment owners receive sale deeds that say "Flat No. 502, 5th Floor" — but this text description has **no geospatial identity**
- If Flat 501 and 502 have a boundary dispute, there is no official 3D geometry to resolve it
- When a bank mortgages Flat 502, it records a lien against "Survey 140/1" — technically encumbering the entire plot
- If the builder fraudulently sells the same unit to two buyers, there is no spatial collision detection
- Municipal authorities estimating tax liability for individual units have no volumetric data
- Emergency responders cannot identify specific floors/rooms in a building information system

CURRENT WORLD vs. WORLD WITH THIS PROJECT

Dimension	Current World (2D Only)	With 3D ULPIN Platform
Property Identity	"Flat 502, 5th Floor, Plot 140/1" (text description)	HSTL01-F5-502-4S (georeferenced 3D volume with precise coordinates)
Ownership Proof	Deed referencing flat number, no spatial verification	Strata title linked to exact 3D bounding box (x,y,z bounds + UDS fraction)
Boundary Disputes	Manual survey, court cases	Automated Shapely 3D topology validation: 0 collisions
Mortgage Specificity	Lien on entire plot	Lien on specific 3D ULPIN unit with conveyance block
Fraud Detection	None (same unit sold twice goes undetected)	Primary key uniqueness + encumbrance lock blocks double sale
Carpet Area Verification	Self-declared by builder	Computed from verified CAD geometry: width × depth = carpet_area_sqm
Emergency Response	Building address only	Room-level 3D coordinates with elevation
Property Tax	Estimated, approximate	Precise volume-based assessment possible
International Compliance	None	ISO 19152 LADM v2 alignment (LA_SpatialUnit, LA_BAUnit, LA_Party, LA_RRR)

SECTION 3 — PROBLEM STATEMENT

1. Primary Problem

India's cadastral system operates exclusively in 2D, making it fundamentally incapable of uniquely identifying, validating, or administering property rights in multi-storey buildings where multiple ownership parcels share identical surface coordinates.

2. Secondary Problems

#	Secondary Problem	Manifestation
SP1	No standardized 3D spatial identifier exists	Each state, city, and builder uses arbitrary flat numbering
SP2	No automated extraction of room boundaries	Manual surveying is expensive, slow, and error-prone
SP3	No topology validation for vertical parcels	Overlapping or duplicate 3D claims go undetected
SP4	No linkage between spatial geometry and legal rights	Deeds and spatial records are disconnected
SP5	No mechanism to prevent encumbered property transfers	Mortgaged units can be illegally sold
SP6	No volumetric visualization for stakeholders	Decision-makers cannot inspect building internals
SP7	No point cloud / photogrammetry validation pipeline	Stated building dimensions cannot be cross-verified

3. Root Causes

Root Cause	Why It Exists	Consequence
Historical 2D paradigm	Land records were designed for agricultural parcels (flat fields)	Multi-storey urban structures are treated as monolithic single parcels
Absence of technical standards	No Indian standard for 3D cadastre exists	Every system is ad-hoc, non-interoperable
Manual survey dependency	Digitization of building interiors requires skilled surveyors	Cost and time prohibit nationwide coverage
Disconnected databases	Survey records, registration records, and revenue records are separate	No single system can associate geometry → rights → encumbrances
No CAD/BIM ingestion pipeline	Government IT systems cannot ingest architectural drawings	Building design data is locked in paper files

4. Existing System Limitations

- **DILRMP/ULPIN**: Only supports 2D surface parcels; no Z-axis, no floor concept, no unit-level granularity
- **Bhunaksha GIS**: State government mapping portal showing 2D survey boundaries; no 3D extrusion
- **Mahabhulekh**: Text-based land record viewer (7/12 extracts); no spatial data
- **CERSAI**: Central Registry of Securitisation — records mortgage liens but has no spatial reference; text-based
- **Sub-Registrar Systems**: Scan and store deed documents as images; no structured 3D data

5. Technical Challenges

Challenge	Complexity	Impact
Accurate extraction from diverse CAD formats	High — vector PDFs vary in layer structure, coordinate systems, and metadata	Extraction accuracy directly affects ULPIN validity
OCR on scanned/photographed floor plans	High — handwritten labels, low resolution, perspective distortion	Manual correction needed for ~10-20% of reads
Precise georeferencing without survey-grade GNSS	Medium — phone-grade GPS has ±5m accuracy	Centroid positions are approximate
Scalable 3D collision detection	Medium — $O(N^2)$ per floor; tolerable for single buildings, challenging at city scale	Need spatial indexing for production
Standardizing across diverse building typologies	High — residential, commercial, industrial, underground all have different layouts	Rule engine must be flexible enough

6. User Challenges

- Citizens cannot verify their apartment's exact boundaries
- Banks cannot definitively identify the unit being mortgaged
- Courts lack spatial evidence for boundary disputes

7. Operational Challenges

- No trained workforce for 3D cadastral surveying

- No government API for programmatic land record access
- Data standards vary across states

8. Scalability Challenges

- 620 million surface parcels × average 3-5 floors = potential billions of 3D units
- Real-time topology validation at national scale requires distributed computing
- Point cloud storage for photogrammetry data requires significant infrastructure

SECTION 4 — PROJECT PURPOSE

Why Was This Project Created?

This project was created in direct response to **Smart India Hackathon Problem Statement 26011**, issued by the Department of Land Resources (DoLR), Ministry of Rural Development. The PS explicitly asks for:

- 3D ULPIN generation
- Vertical property mapping
- Multi-storey apartment delineation
- Underground infrastructure representation
- Topology validation
- Integration with existing land record systems

Mission Statement

To build India's first open, standards-compliant 3D cadastral platform that transforms flat 2D land records into volumetric spatial registries, enabling precise identification and legal administration of every room, corridor, and utility space in multi-storey buildings.

Vision Statement

A future where every property unit in India — from a 500-sq-ft apartment on the 15th floor to an underground parking bay — has a unique, immutable, government-recognized 3D spatial identity linked to verified ownership rights.

Long-Term Objective

Nationwide deployment of the 3D ULPIN framework as a standard extension to the DILRMP/ULPIN programme, integrated with state revenue systems, Sub-Registrar offices, CERSAI, RERA portals, and Smart City Command Centers.

Short-Term Objectives

Objective	Timeline
Demonstrate working prototype on real building data	☐ Completed (700 parcels, IIIT Nagpur Hostel)
Pass all SIH PS 26011 requirements	☐ Completed (full traceability matrix)
Support multi-building campus registry	☐ Completed (4 buildings registered)
Implement ISO 19152 LADM v2 RRR engine	☐ Completed (allotment, transfer, encumbrance)
Build generalized ingestion pipeline for new buildings	☐ Completed (config-driven, zero code changes)
Add drone photogrammetry ML height validation	☐ Completed (unsupervised DBSCAN pipeline)

What Does Success Look Like?

1. A government surveyor can upload a building's floor plan PDF and receive 3D ULPINs for every room — automatically
2. A citizen can search their apartment by 3D ULPIN and see its exact geometry, area, volume, and ownership status
3. A bank can stamp a mortgage lien against a specific 3D unit and prevent its sale until cleared

- 4. A court can resolve a boundary dispute by inspecting the 3D topology validation report
- 5. A municipal corporation can assess property tax based on precise volumetric data

SECTION 5 — TARGET USERS AND STAKEHOLDERS

User Personas

Persona 1: Rajesh — Land Revenue Officer (Tehsildar)

Attribute	Detail
Background	State government officer managing land records for a taluka
Role	Maintains revenue records (7/12 extracts), processes mutations
Current Problem	Multi-storey buildings appear as single 2D parcels; cannot resolve vertical ownership queries
Goal	Generate official 3D identifiers for every unit in buildings under his jurisdiction
Current Solution	Manual register entries with text descriptions ("Flat 3, 2nd Floor")
Pain Points	Frequent disputes, no spatial evidence, slow manual processes
How Platform Helps	Upload floor plan → automated 3D ULPIN generation → topology validation → strata title registry
Most Useful Features	Ingestion pipeline, topology validation, analytics dashboard

Persona 2: Priya — Home Buyer / Apartment Owner

Attribute	Detail
Background	IT professional who purchased a 3BHK apartment
Role	Property owner seeking clear title and boundary certainty
Current Problem	Deed says "Flat 802, 8th Floor" but no way to verify exact boundaries or carpet area
Goal	Know the exact 3D dimensions of her property and confirm no overlapping claims
Current Solution	Relies on builder's declaration; no independent verification
Pain Points	Builder disputes, carpet area vs. super area confusion, no RERA-verifiable geometry
How Platform Helps	Search by 3D ULPIN → view exact bounding box, area, volume, UDS fraction → verify encumbrance status
Most Useful Features	3D parcel search, digital twin visualization, encumbrance check

Persona 3: Anil — Branch Manager, State Bank of India

Attribute	Detail
Background	Bank officer processing home loan applications
Role	Evaluates property for mortgage, stamps lien, ensures no prior encumbrances
Current Problem	Mortgage lien is recorded against the <i>plot</i> , not the specific apartment unit
Goal	Register a mortgage specifically against Unit HSTL01-F5-502-4S, not the entire Survey 140/1
Current Solution	Text-based CERSAI filing with no spatial reference
Pain Points	Double-mortgage fraud undetectable; cannot block illegal resale of encumbered unit
How Platform Helps	Stamp lien via API → automatic conveyance block on that specific 3D ULPIN → release upon loan satisfaction
Most Useful Features	Encumbrance stamping API, anti-fraud conveyance block, mutation audit log

Persona 4: Dr. Meena — GIS Surveyor / Architect

Attribute	Detail
Background	Licensed surveyor contracted by the municipality to digitize building records
Role	Converts paper/digital floor plans into structured cadastral data
Current Problem	No standardized tool to ingest CAD drawings and produce 3D cadastral records
Goal	Upload floor plans for multiple buildings and generate validated 3D datasets efficiently
Current Solution	Manual GIS digitization in QGIS/AutoCAD — one building takes weeks
Pain Points	Time-consuming, error-prone, no validation, no standardized output format
How Platform Helps	Upload PDF → automated extraction (vector or CV+OCR) → 3D stacking → topology audit → CSV + DB output

Attribute	Detail
Most Useful Features	Ingestion API, dual-mode extraction, point cloud cross-validation, building configuration system

Persona 5: Commissioner — Municipal Corporation Smart City Cell

Attribute	Detail
Background	Urban planning officer overseeing building approvals and property tax
Role	Verifies building compliance, assesses property tax, plans infrastructure
Current Problem	No volumetric data for buildings; tax assessed on estimated area
Goal	Access precise carpet area and volume data for every unit in every building
Current Solution	Self-assessment forms from owners; no independent spatial verification
Pain Points	Revenue leakage, inaccurate assessments, no digital twin for planning
How Platform Helps	Analytics API provides total carpet area, volume, parcel counts per building; 3D digital twin for visual inspection
Most Useful Features	Analytics dashboard, 3D digital twin, building registry

Stakeholder Analysis

Stakeholder	Role	Benefit	Interaction	Influence on Success
DoLR / Ministry	Problem statement issuer, policy authority	National 3D cadastre capability	Sets standards, provides mandate	Critical — policy adoption required
State Revenue Departments	Operational deployment authorities	Efficient land administration	Deploy and operate the system	Critical — ground-level implementation
NIC / DILRMP	Technology partner	Integration with existing ULPIN systems	API integration, data exchange	High — technical interoperability
RERA Authorities	Regulatory oversight	Carpet area verification capability	Query API for compliance checks	Medium — regulatory validation
CERSAI	Central encumbrance registry	Spatial specificity for liens	API integration for mortgage records	Medium — financial integration
IIIT Nagpur	Prototype host institution	Campus digitization, academic contribution	Provides real-world test data	Medium — prototype validation
SIH Evaluators	Assessment panel	Evaluation of technical innovation	Review and scoring	High — immediate evaluation

SECTION 6 — EXISTING SOLUTIONS AND GAP ANALYSIS

Comparison Table

Solution	Approach	Strengths	Weaknesses	Technology	Gap Filled by Our Project
DILRMP / ULPIN	14-digit surface parcel ID	National standard, government-backed	2D only; no vertical representation	GIS + alphanumeric	3D extension with floor/unit granularity
Bhunaksha	State GIS parcel map viewer	Spatial visualization of survey boundaries	No building internals, no 3D	Web GIS	3D building internals + volumetric data
Mahabhulekh	Text-based land record viewer	Digitized 7/12 extracts	No spatial reference; text only	Web portal	Spatial geometry linked to records
CERSAI	Text-based encumbrance registry	National mortgage lien register	No spatial specificity; plot-level only	Database	Unit-level 3D encumbrance with spatial reference
Google Maps 3D	Photogrammetric 3D city models	Visual 3D representation	No room-level detail; no cadastral data; commercial	Photogrammetry + ML	Room-level 3D parcels with legal metadata
AutoCAD / Revit BIM	Professional CAD/BIM tools	Precise architectural modeling	Design tools, not cadastral systems; no rights registry	Proprietary CAD	Cadastral layer on top of CAD data
European 3D Cadastre (Netherlands)	Legal 3D registration (Kadaster)	Operational 3D cadastre	Country-specific; not adaptable to Indian legal framework	Custom GIS	Indian-specific ISO 19152 implementation

What This Project Does Differently

1. **Bridges 2D ULPIN → 3D:** Extends the existing national framework rather than replacing it. The parent surface ULPIN (pu-id: 33550994106) is preserved as the root.
2. **Automated Extraction:** Uses CV + OCR + vector PDF parsing to extract room boundaries from existing floor plans — no new hardware required.
3. **Zero-Code Building Onboarding:** The `BuildingConfig` JSON + `RoomTypeRule` pattern allows new buildings to be added via configuration, not code changes.
4. **ISO 19152 LADM v2 Compliance:** Implements the international standard's core concepts (`LA_SpatialUnit`, `LA_BAUnit`, `LA_Party`, `LA_RRR`).
5. **Anti-Fraud Mechanisms:** Active encumbrance check blocks conveyance; capacity limits prevent overcrowding; immutable audit log provides transaction trail.

*[!IMPORTANT] We do not claim superiority over mature European 3D cadastre systems (Netherlands Kadaster, Singapore SLA). Our innovation is in adapting 3D cadastre concepts to the **Indian administrative context** using **automated extraction from existing floor plans** — addressing the scale and data availability challenges unique to India.*

SECTION 7 — PROPOSED SOLUTION

Core Concept

A **config-driven, dual-mode extraction pipeline** that converts architectural floor plans (vector CAD PDFs or scanned images) into georeferenced, topologically validated, rights-linked 3D volumetric parcels — each assigned a standardized 3D ULPIN and managed through an ISO 19152-compliant REST API.

Key Innovation

The Rule Engine architecture: Rather than hardcoding building-specific logic, the system uses a declarative JSON configuration (`BuildingConfig` + `RoomTypeRule[]`) to define how rooms are classified. A `RuleEngine` evaluates each detected boundary against label patterns (regex), shape conditions (width/height), and spatial bands (Y-coordinate zones) to assign room types, property IDs, and depth values. This means any building in India can be onboarded by creating a JSON file — zero Python changes.

End-to-End Lifecycle

```
USER ENTERS THE SYSTEM
↓
User accesses the DoLR-style bilingual landing page (English/Hindi)
↓
USER PROVIDES INPUT
↓
Option A: Surveyor uploads floor plan PDF + building config via /api/ingestion/run
Option B: Citizen searches existing parcels via Mahabhulekh-style cascading search (District → Taluka → Village → Survey)
↓
SYSTEM PROCESSES DATA
↓
1. Auto-detect file type (vector PDF vs. scanned image)
2. If vector: PyMuPDF extracts drawing paths + text labels → deduplicated rectangles
3. If scanned: OpenCV adaptive threshold → morphological closing → contour detection → Tesseract OCR
4. RuleEngine evaluates each detected boundary → assigns type, prop_id, depth
5. Coordinate transform: CAD pixels → metric meters → WGS84 lat/lon
↓
BACKEND LOGIC EXECUTES
↓
6. Multi-floor vertical stacking: each base unit is replicated across N floors
7. Z-coordinates computed: z_min = (floor-1) × pitch, z_max = z_min + room_height
8. 3D ULPIN generated: BUILDING-FLOOR-UNIT-TYPE
9. Carpet area = width × depth; Volume = area × height; UDS = area / total_building_area
↓
DATABASE / EXTERNAL SERVICES INTERACT
↓
10. Parcels inserted into SQLite parcels_3d table
11. Building metadata updated (total_floors, data_status = "completed")
12. CSV dataset exported for GIS analysis
↓
AI / ALGORITHMS PROCESS DATA
↓
13. Topology Validator: Shapely box() pairwise intersection on same floor → Z-interval check → collision report
14. Point Cloud Validator (if .ply/.las uploaded): Z-histogram → peak detection → DBSCAN clustering → floor count verification
↓
RESULT IS GENERATED
↓
15. Ingestion result JSON: total_units, topology_validation, ocr_diagnostics, point_cloud_audit
16. 3D mesh data streamed via /api/parcels/mesh-data/{building_id}
↓
USER RECEIVES OUTPUT
↓
17. Surveyor: receives ingestion report with validation results and sample ULPIN
18. Citizen: views 3D digital twin in WebGL, hovers to inspect unit details
19. Bank: stamps mortgage lien via POST /api/encumbrances/stamp
20. Officer: runs mutation via POST /api/rights/allot or /api/rights/transfer
```

SECTION 8 — SYSTEM ARCHITECTURE

Architecture Layers

1. Presentation Layer

- **Purpose:** Bilingual (English/Hindi) user interface for public access
- **Components:** index_en.html, index_hi.html — DoLR/Mahabhulekh-styled government portal
- **Technology:** Static HTML/CSS, responsive design
- **Communication:** Serves as entry point → links to Application layer

2. Frontend Application Layer

- **Purpose:** Interactive cadastral application with 3D visualization
- **Components:** `app.html` + `app.js` + `app.css` — Mahabhulekh cascading search, building selector cards, 3D digital twin viewer, title inspector, mutation console
- **Technology:** Vanilla JavaScript, Plotly.js (WebGL), CSS
- **Inputs:** User search queries, building selection, mutation commands
- **Outputs:** Rendered 3D mesh, parcel details, mutation confirmations
- **Communication:** REST JSON calls to Backend API Layer

3. API Gateway Layer

- **Purpose:** RESTful API surface for all cadastral operations
- **Components:** FastAPI application with CORS middleware, OpenAPI 3.0 auto-documentation
- **Technology:** FastAPI 0.110+, Uvicorn ASGI server
- **Inputs:** HTTP requests (GET/POST) with query parameters, form data, file uploads
- **Outputs:** JSON responses conforming to Pydantic response models
- **Communication:** Routes requests to appropriate router → service layer

4. Router Layer (API Routing)

- **Purpose:** Modular organization of API endpoints by domain
- **Components:** 8 routers — `institutions`, `buildings`, `parcels`, `rights`, `encumbrances`, `topology`, `analytics`, `ingestion`
- **Technology:** FastAPI `APIRouter`
- **Communication:** Each router → specific service layer modules

5. Business Logic / Services Layer

- **Purpose:** Core domain logic and processing engines
- **Components:**
 - `cadastral_engine.py` — End-to-end ingestion pipeline orchestrator
 - `cadastral_config.py` — `BuildingConfig` + `RuleEngine` for declarative room classification
 - `vector_extractor.py` — PyMuPDF-based vector CAD PDF extraction
 - `cv_extractor.py` — OpenCV + Tesseract OCR scanned image extraction
 - `topology_validator.py` — 3D Shapely collision detection
 - `rights_manager.py` — ISO 19152 RRR allotment, transfer, encumbrance management
 - `pointcloud_validator.py` — ML-based height validation from drone data
- **Technology:** Python 3.9+, Shapely 2.0, OpenCV, PyMuPDF, Tesseract, scikit-learn, scipy
- **Communication:** Receives processed data from routers → queries/writes database → returns results

6. AI / ML Layer

- **Purpose:** Computer vision extraction + unsupervised height validation
- **Components:** `cv_extractor.py` (contour detection, OCR), `pointcloud_validator.py` (DBSCAN clustering, peak detection)
- **Technology:** OpenCV, Tesseract OCR, scikit-learn DBSCAN, scipy `find_peaks`
- **Communication:** Called by `cadastral_engine.py` during ingestion pipeline

7. Database Layer

- **Purpose:** Persistent storage for all cadastral entities and transactions
- **Components:** SQLite database `cadastr_3d.db` with 7 tables
- **Technology:** SQLite 3, Python `sqlite3` module
- **Communication:** Accessed by services layer via `get_db_connection()`

8. Configuration Layer

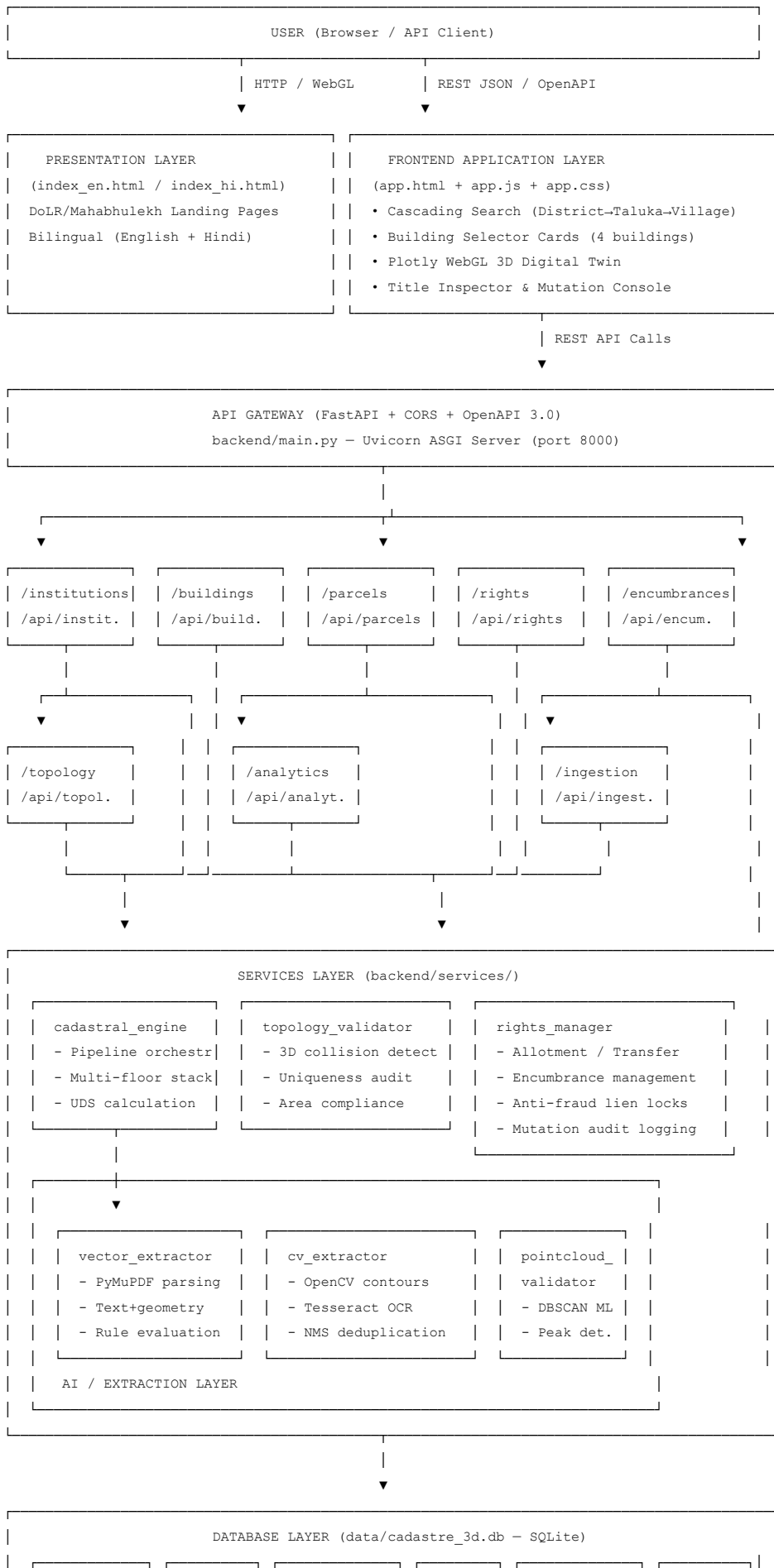
- **Purpose:** Declarative building specifications and institutional registry
- **Components:** `config/institutional_registry.json`, `config/buildings/*.json`
- **Technology:** JSON files, Pydantic validation
- **Communication:** Loaded by `cadastral_engine.py` and `seed_data.py`

9. Static File / Frontend Serving Layer

- **Purpose:** Serve frontend assets

- **Components:** FastAPI `StaticFiles` mount at `/static`
- **Technology:** FastAPI static file middleware
- **Communication:** Serves HTML/CSS/JS files to browser

Text-Based Architecture Diagram



Technology	Category	Purpose	Where Used	Why Chosen	Alternatives	Advantages	Limitations
Tesseract (pytesseract)	OCR	Room number/label extraction	cv_extractor.py	Open-source, multilingual	Google Vision API, AWS Textract	Free, offline, good for printed text	~10-20% error on architectural drawings
scikit-learn 1.6+	ML	DBSCAN clustering for floor level detection	pointcloud_validator.py	Standard ML library	TensorFlow, PyTorch	Simple API, well-documented	Not deep learning
scipy 1.13+	Scientific	Peak detection in Z-histogram	pointcloud_validator.py	Signal processing	NumPy manual	find_peaks is purpose-built	Statistical, not ML

GEOSPATIAL

Technology	Category	Purpose	Where Used	Why Chosen	Alternatives	Advantages	Limitations
Shapely 2.0+	Computational Geometry	3D collision detection, polygon intersection	topology_validator.py, validate_properties.py, cadastral_engine.py	Fastest Python geometry library (GEOS backend)	GeoPandas only, PostGIS	Pure computational geometry, no DB dependency	2D operations extended to 3D manually
PyMuPDF (fitz) 1.24+	PDF Processing	Vector CAD PDF parsing	vector_extractor.py, cv_extractor.py, cadastral_engine.py	Extracts exact vector geometry from PDFs	pdfplumber, camelot	Access to drawing paths, text, rects	Memory-intensive for large PDFs
pyproj 3.6+	Geodesy	Coordinate reference system transforms	Requirements (available for advanced georeferencing)	Industry standard for CRS	Manual math	PROJ library bindings	Not fully utilized in prototype (manual math used)
Pandas 2.0+	Data Analysis	Tabular data manipulation	Data processing scripts	Standard data analysis	Pure CSV	Rich operations	Overhead for simple CSV
GeoPandas 1.0+	GeoData	Spatial data frames	Available for analysis	Extends Pandas with geometry	Manual	Spatial joins, exports	Heavy dependency
laspy 2.6+	Point Cloud	LAS/LAZ file reading	pointcloud_validator.py	Standard LiDAR format reader	open3d, pdal	Lightweight, focused	LAS format only

CAD EXTRACTION

Technology	Category	Purpose	Where Used	Why Chosen	Alternatives	Advantages	Limitations
PyMuPDF (fitz)	CAD Parsing	Extract vector drawings from PDF floor plans	vector_extractor.py	Direct access to PDF drawing objects	pdfminer, pikepdf	get_drawings() returns exact geometric paths	PDF structure varies by CAD software
Pillow (PIL) 10.0+	Image Processing	Image format handling, fallback loading	cv_extractor.py	Universal image format support	Pure OpenCV	Broad format support	Not used for primary processing

DEVOPS / VERSION CONTROL

Technology	Category	Purpose	Where Used	Why Chosen	Alternatives	Advantages	Limitations
Git	VCS	Source code management	Repository root	Industry standard	SVN, Mercurial	Distributed, branching	—
GitHub	Hosting	Code hosting + collaboration	github.com/adiscripts03/3D_UPLIN_SIH	Free, CI/CD integration	GitLab, Bitbucket	Issues, PRs, Actions	—
pip + venv	Package Mgmt	Dependency management	requirements.txt, ulpin_env/	Standard Python	conda, poetry	Simple, universal	No lock file

SECTION 10 — TECHNOLOGY DEEP DIVE

10.1 FastAPI — Backend REST Framework

What it is: A modern, high-performance Python web framework for building APIs, based on standard Python type hints with automatic OpenAPI documentation.

Why it is relevant: The cadastral platform needs a modular REST API to serve 8 distinct domain routers (institutions, buildings, parcels, rights, encumbrances, topology, analytics, ingestion) — each with different request/response schemas. FastAPI's router pattern, Pydantic model integration, and auto-generated Swagger UI make it ideal.

Where it is used:

- `backend/main.py`: Application initialization, CORS middleware, router registration, static file mounting, route handlers for landing pages (`/`, `/en`, `/hi`, `/app`, `/portal`, `/twin`)
- `backend/routers/*.py`: 8 API routers with typed endpoint definitions
- `backend/models.py`: 10 Pydantic models for request/response validation

How it integrates:

- **Database:** Each router handler calls `get_db_connection()` to execute SQL queries
- **Services:** Routers delegate business logic to `services/` modules (e.g., `rights.py` calls `rights_manager.allot_occupant_to_unit()`)
- **Frontend:** Serves `frontend/` as static files via `StaticFiles` mount; returns `FileResponse` for HTML pages

Input → Processing → Output:

- Input: HTTP requests with path params, query params, form data, or file uploads
- Processing: Pydantic validates inputs → router handler → service logic → database query → result construction
- Output: JSON response with appropriate HTTP status code and content-type headers

Why chosen over alternatives: | Criterion | FastAPI | Flask | Django | |---|---|---| | Auto API docs | ☐ Swagger + ReDoc | ☐ Manual | ☐ Manual | | Type validation | ☐ Pydantic built-in | ☐ Manual | Partial (DRF) | | Async support | ☐ Native | ☐ Limited | ☐ Limited | | Learning curve | Low | Low | High | | File upload handling | ☐ UploadFile + File() | Manual | DRF Parsers |

10.2 Shapely 2.0 — Computational Geometry / Topology Engine

What it is: A Python library for manipulation and analysis of planar geometric objects, backed by the GEOS C library.

Why it is relevant: The core innovation of this project — 3D topology validation — depends on computing whether any two volumetric parcels spatially overlap. Shapely provides the geometric primitives (`box()`, `.intersection()`, `.area`) needed for this.

Where it is used:

- `backend/services/topology_validator.py`: The `validate_building_topology()` function constructs 2D Shapely boxes for each parcel, then performs pairwise intersection tests within each floor group
- `validate_properties.py`: Standalone validation script performing the same collision detection + cross-floor elevation checks
- `backend/services/cadastral_engine.py`: Topology audit within the ingestion pipeline

How 3D collision detection works with a 2D library: Since Shapely is fundamentally 2D, the system implements 3D detection as a two-stage process:

1. **Z-interval check:** `z_overlap = min(p1.z_max, p2.z_max) - max(p1.z_min, p2.z_min)` — if ≤ 0 , parcels are on different vertical levels, skip
2. **2D polygon intersection:** `b1.intersection(b2).area` — if $> 0.05 \text{ m}^2$ (tolerance for boundary adjacency), flag as collision
3. **3D volume:** `volume_overlap = intersection_area * z_overlap`

Why chosen over PostGIS: PostGIS provides native 3D geometry operations, but requires a PostgreSQL server. For a prototype that must run locally with zero configuration, Shapely + SQLite provides the same geometric capability without infrastructure overhead.

10.3 PyMuPDF (fitz) — Vector CAD PDF Extraction

What it is: A Python binding for the MuPDF library, providing access to PDF document structure including vector drawings, text, and images.

Why it is relevant: The primary data source for this system is architectural CAD floor plans in PDF format. PyMuPDF's `get_drawings()` method returns the exact geometric drawing objects (rectangles, paths, lines) embedded by CAD software, preserving precise coordinates without rasterization.

Where it is used:

- `backend/services/vector_extractor.py: extract_units_from_vector_pdf()` opens the PDF, calls `page.get_drawings()` and `page.get_text("words")`, deduplicates rectangles, matches text labels to containing rectangles, and applies the `RuleEngine` for room classification
- `backend/services/cv_extractor.py: Used as a fallback renderer` — `page.get_pixmap(dpi=300)` renders raster PDFs to images for CV processing
- `backend/services/cadastral_engine.py: detect_source_type()` uses `fitz` to determine if a PDF is vector (many drawings + words) or scanned (few/no drawings)

Input → Processing → Output:

- Input: `data/floor_plan.pdf` (architectural CAD PDF)
- Processing: Opens PDF → extracts page → gets all drawing objects → filters by size (removes >600px wide outer boundaries) → deduplicates (0.5pt tolerance) → matches text words to rectangle interiors → applies coordinate scaling (0.1362 m/pt) → evaluates rules
- Output: List of base unit dictionaries with `prop_id`, `type`, `real_width_m`, `real_depth_m`, `real_x_start_m`, `real_x_end_m`, etc.

10.4 OpenCV + Tesseract OCR — Computer Vision Extraction

What it is: OpenCV is the world's most widely used computer vision library. Tesseract is an open-source OCR engine maintained by Google.

Why it is relevant: Not all floor plans are vector CAD PDFs. Many are scanned paper drawings or photographs. The CV pipeline extracts room boundaries from raster images and reads room numbers via OCR.

Where it is used: `backend/services/cv_extractor.py`

Processing Pipeline:

1. `load_image_from_source()`: Loads image or renders PDF page to pixel array at 300 DPI
2. `detect_room_contours()`:
 - Convert to grayscale → Gaussian blur (5×5)
 - Adaptive threshold (Gaussian, block=15, C=3) → inverts to highlight walls
 - Morphological closing (3×3 kernel, 2 iterations) → bridges wall gaps
 - `cv2.findContours(RETR_TREE, CHAIN_APPROX_SIMPLE)` → all contours with hierarchy
 - Filter by area (0.15% to 70% of total image area) → removes noise and outer perimeter
 - Filter by aspect ratio (reject if > 15:1) → removes thin wall artifacts
 - Non-maximum suppression (65% IoU threshold) → deduplicates overlapping detections
3. `extract_ocr_text_and_confidence()`: Crops ROI → `pytesseract.image_to_data()` → returns text + per-word confidence scores
4. Results: Each detected room gets `prop_id`, `type`, coordinates, and `ocr_confidence` score
5. Low-confidence warnings (< 60%) flagged in diagnostics report

Limitations openly stated:

- OCR accuracy on architectural drawings is ~80-90% — manual correction needed for 10-20% of reads
- Contour detection parameters may need tuning for different drawing styles
- No deep learning segmentation model yet (stretch goal)

10.5 SQLite — Spatial Cadastral Database

What it is: A self-contained, serverless, zero-configuration SQL database engine.

Why it is relevant: The prototype needs a relational database to store 7 interlinked tables (institutions, buildings, parcels_3d, parties, strata_titles, encumbrances, mutation_audit_log) with foreign key relationships and indexed queries — but without requiring database server installation.

Where it is used: `backend/database.py` defines `init_db()` (7 CREATE TABLE statements), `get_db_connection()` (returns connection with PRAGMA foreign_keys = ON), and `DB_PATH = data/cadastre_3d.db`

Schema design choices:

- `parcels_3d.ulpin_3d TEXT PRIMARY KEY` — the 3D ULPIN is the primary key, not an auto-increment ID, because the ULPIN is the canonical identifier
- FOREIGN KEY (`building_id`) REFERENCES `buildings(building_id)` ON DELETE CASCADE — cascade deletion ensures referential integrity
- Indexes on `parcels_3d(building_id)` and `parcels_3d(building_id, floor)` for efficient spatial queries
- `is_common_property INTEGER (0/1)` because SQLite lacks native BOOLEAN

10.6 Plotly (WebGL) — 3D Digital Twin Rendering

What it is: A graphing library that provides interactive, publication-quality graphs with WebGL hardware acceleration for 3D.

Why it is relevant: The system needs to render 700+ volumetric parcels as interactive 3D boxes that users can rotate, zoom, filter by floor, and hover to inspect metadata.

Where it is used:

- `visualize_3d.py`: Generates static HTML 3D twin (`hostel_3d_twin.html`) with all 700 mesh boxes pre-rendered
- `frontend/app.js`: Dynamically constructs Plotly 3D mesh from `/api/parcels/mesh-data/{building_id}` JSON response

How each 3D parcel is rendered: Each parcel is a `Mesh3d` trace with 8 vertices (cuboid corners) and 12 triangular facets (2 per face × 6 faces). Colors encode status: green (vacant), amber (partial), red (full), cyan (washroom), grey (stairwell/lift), purple (bridgeway), blue (corridor), amber (common hall).

10.7 scikit-learn / scipy — ML Height Validation

What it is: scikit-learn provides ML algorithms; scipy provides signal processing tools.

Where it is used: `backend/services/pointcloud_validator.py`

Processing pipeline:

- Load point cloud Z-coordinates from `.ply`, `.xyz`, `.las`, or `.obj` files
- Remove ground noise (1st-99th percentile filtering)
- Normalize to ground level = 0.0
- Compute Z-histogram at 0.1m resolution
- `scipy.signal.find_peaks()` detects density peaks (floor slabs create horizontal point concentrations)
- Calculate inter-peak distances → estimate floor pitch
- Compare detected floor count and pitch against `BuildingConfig` values
- Flag discrepancy if floor count differs by >1 or pitch differs by >0.45m

Why this approach: Fully unsupervised — no training data needed. Works with any point cloud format. Degrades gracefully (returns "SKIPPED" if no point cloud provided).

SECTION 11 — FEATURE BREAKDOWN

Feature 1: Dual-Mode Floor Plan Ingestion

Attribute	Detail
Feature Name	Automated Floor Plan Ingestion (Vector + Scanned)
Purpose	Extract room boundaries from any architectural floor plan format
Target User	Surveyors, Architects, Revenue Officers
Problem Solved	Manual digitization is slow and expensive
Input	PDF (vector CAD or scanned), image files (.png, .jpg)
Processing	Auto-detect type → vector extraction (PyMuPDF) or CV extraction (OpenCV + OCR) → RuleEngine classification → coordinate scaling
Technology	PyMuPDF, OpenCV, Tesseract OCR, Pydantic BuildingConfig
Output	List of base unit records with dimensions, type, coordinates
Edge Cases	Mixed vector/raster PDFs; handwritten labels; non-standard CAD exports
Failure Scenarios	Empty PDF pages; no contours detected; OCR misreads (flagged in diagnostics)
Future Improvements	Deep learning room segmentation (Mask R-CNN/SAM); DXF/DWG native support; IFC/BIM ingestion

Feature 2: 3D ULPIN Generation

Attribute	Detail
Feature Name	Standardized 3D Unique Land Parcel Identification Number Generation
Purpose	Assign every room a globally unique, human-readable, machine-parseable 3D identifier
Target User	All stakeholders
Problem Solved	No standardized 3D spatial identifier exists in India
Input	Building code, floor number, unit ID, zoning type

Attribute	Detail
Processing	Schema: {BUILDING_CODE}-F{FLOOR}-{UNIT_ID}-{ZONING_TYPE}
Technology	Python string formatting, CSV I/O
Output	e.g., HSTL01-F5-501-4S, HSTL01-F3-BRIDGE1-BRIDGE
Edge Cases	Special characters in room IDs (& → AND, spaces → underscores)
Failure Scenarios	Duplicate ULPINs detected → flagged by topology validator
Future Improvements	Integrate with national DILRMP 14-digit ULPIN as prefix; add check digit

Feature 3: 3D Topology Validation

Attribute	Detail
Feature Name	3D Volumetric Collision Detection & Compliance Audit
Purpose	Ensure no two parcels occupy the same 3D space
Target User	Surveyors, Revenue Officers, Courts
Problem Solved	Overlapping property claims go undetected
Input	All parcels for a building (from database)
Processing	Group by floor → pairwise Z-interval check → Shapely 2D box intersection → area threshold (0.05 m ²) → report
Technology	Shapely 2.0 (GEOS backend)
Output	JSON report: total parcels audited, duplicates, area violations, collision errors, compliance score
Edge Cases	Halls exempt from collision check (intentional overlap with corridor); boundary adjacency tolerance
Failure Scenarios	Incorrect elevation data → false collisions between floors
Future Improvements	R-tree spatial indexing for O(N log N); 3D boolean operations; tolerance configuration per building type

Feature 4: ISO 19152 LADM v2 Rights Registry

Attribute	Detail
Feature Name	Strata Title Allotment, Conveyance Mutation & Anti-Fraud Engine
Purpose	Register, transfer, and protect property rights at 3D unit level
Target User	Revenue Officers, Property Owners, Banks, Sub-Registrars
Problem Solved	No mechanism links spatial geometry to legal rights with fraud prevention
Input	ULPIN, party ID, right type, conveyance details
Processing	Verify parcel exists → check common property lock → verify capacity → check encumbrance block → insert title → log mutation
Technology	SQLite, Python business logic
Output	Success/failure with message, occupancy status, mutation ID
Edge Cases	Party already holds title; parcel at max capacity; unit is common property (inalienable)
Failure Scenarios	Active mortgage lien blocks transfer (by design — anti-fraud feature)
Future Improvements	Aadhaar-based party verification; digital signature on mutations; blockchain audit trail

Feature 5: Mortgage Lien Stamping & Conveyance Block

Attribute	Detail
Feature Name	Bank Mortgage Encumbrance Registry with Automatic Conveyance Block
Purpose	Allow banks to register liens against specific 3D units and prevent illegal resale
Target User	Banks, Financial Institutions, Sub-Registrars
Problem Solved	Current CERSAI system has no spatial specificity; mortgaged plots can be resold
Input	ULPIN, mortgagee name, sanction reference, loan amount
Processing	Verify parcel → check duplicate sanction ref → insert encumbrance → log mutation
Technology	SQLite, rights_manager.py
Output	Confirmation with lien details
Edge Cases	Duplicate sanction reference (blocked); lien on common property (allowed for institutional mortgage)
Failure Scenarios	Invalid ULPIN
Future Improvements	CERSAI API integration; real-time NOC verification; SBI/BOB/PNB direct integration

Feature 6: Interactive 3D Digital Twin

Attribute	Detail
Feature Name	WebGL 3D Volumetric Cadastre Visualization
Purpose	Visual inspection of building internals with per-unit metadata

Attribute	Detail
Target User	All stakeholders
Problem Solved	Cannot visualize multi-storey property boundaries
Input	3D mesh data from <code>/api/parcels/mesh-data/{building_id}</code>
Processing	Construct Plotly Mesh3d traces → apply color coding → add floor labels → configure camera
Technology	Plotly.js (WebGL), JavaScript
Output	Interactive 3D model with hover inspection, floor filtering, rotation/zoom
Edge Cases	Buildings with no digitized data show "pending" message
Failure Scenarios	Browser without WebGL support; mobile performance on 700+ meshes
Future Improvements	Three.js for more control; CesiumJS for globe integration; AR/VR walkthroughs

Feature 7: Drone Photogrammetry ML Cross-Validation

Attribute	Detail
Feature Name	Unsupervised Point Cloud Height Verification
Purpose	Cross-validate stated building dimensions against drone-captured 3D data
Target User	Surveyors, Auditors
Problem Solved	No way to verify if stated floor count/height matches reality
Input	Point cloud file (.ply, .xyz, .las, .obj)
Processing	Load Z-coords → percentile filtering → histogram → peak detection → DBSCAN clustering → compare with config
Technology	scikit-learn DBSCAN, scipy find_peaks, laspy
Output	Verification report: detected floors, estimated pitch, discrepancy flag
Edge Cases	Sparse point cloud (<100 points); very tall buildings (>50 floors); non-standard floor pitches
Failure Scenarios	Gracefully skipped if no point cloud provided
Future Improvements	Automated drone flight path planning; real-time photogrammetry; Gaussian splatting

Feature 8: Config-Driven Building Onboarding

Attribute	Detail
Feature Name	Zero-Code New Building Registration via JSON Configuration
Purpose	Onboard any building by providing a config file, not writing code
Target User	System administrators, surveyors
Problem Solved	Code-level changes needed for each new building type
Input	JSON file with BuildingConfig + RoomTypeRule[]
Processing	Pydantic validates config → RuleEngine initialized with rules → pipeline runs
Technology	Pydantic BaseModel, RuleEngine with regex + shape conditions + Y-bands
Output	Fully processed building with 3D parcels
Edge Cases	Missing rules (fallback to generic room type); conflicting rules (first match wins)
Failure Scenarios	Invalid JSON format; missing required fields (Pydantic raises ValidationError)
Future Improvements	Web-based config builder UI; auto-suggest rules from floor plan analysis

SECTION 12 — COMPLETE USER FLOW

User Flow 1: Surveyor Ingesting a New Building

```
STEP 1: Surveyor navigates to /docs (Swagger UI)
↓
STEP 2: Opens POST /api/ingestion/run
↓
STEP 3: Uploads floor plan PDF + enters building metadata
        (building_id, building_name, total_floors, floor_pitch, anchor GPS, rules JSON)
↓
STEP 4: System auto-detects PDF type (vector vs. scanned)
↓
STEP 5a (Vector): PyMuPDF extracts drawing rectangles + text labels
STEP 5b (Scanned): OpenCV contour detection + Tesseract OCR
↓
STEP 6: RuleEngine classifies each detected boundary → room types assigned
↓
STEP 7: Coordinate scaling (CAD units → meters) + georeferencing (meters → WGS84)
↓
STEP 8: Multi-floor stacking (base units × N floors) with Z-elevation computation
↓
STEP 9: 3D ULPIN generation for each unit
↓
STEP 10: Topology validation → collision detection → uniqueness audit
↓
STEP 11: Point cloud ML validation (if provided)
↓
STEP 12: Data persisted to SQLite + CSV exported
↓
STEP 13: Surveyor receives JSON response:
        {status, total_units, topology_validation, ocr_diagnostics, point_cloud_audit, sample_ulpin}
```

Error Flow: If PDF is empty or no units extracted → 500 error with "No architectural units could be extracted" message

User Flow 2: Citizen Searching Their Property

```
STEP 1: Citizen opens landing page (/ or /en or /hi)
↓
STEP 2: Selects "3D Cadastre Portal" → navigates to /app
↓
STEP 3: Uses Mahabhulekh-style cascading search:
        District: Nagpur → Taluka: Nagpur Rural → Village: Waranga → Survey: 140/1
↓
STEP 4: Building cards displayed (ADMIN01, ACAD01, HSTL01, RES01)
↓
STEP 5: Clicks "Hostel Block A" → fetches GET /api/parcels/mesh-data/HSTL01
↓
STEP 6: 3D digital twin loads in WebGL → 700 colored boxes rendered
↓
STEP 7: Citizen uses floor filter dropdown → selects "Level 5"
↓
STEP 8: Hovers over their unit → tooltip shows:
        ULPIN: HSTL01-F5-501-4S, Area: 33.5 m², Volume: 97.15 m³,
        Occupancy: 2/4, Elevation: 13.6m - 16.5m
↓
STEP 9: Clicks "Title Inspector" → GET /api/parcels/HSTL01-F5-501-4S
↓
STEP 10: Full parcel detail displayed with occupants, capacity, encumbrance status
```

User Flow 3: Bank Stamping a Mortgage Lien

```
STEP 1: Bank officer accesses POST /api/encumbrances/stamp
↓
STEP 2: Submits: ulpin_3d="HSTL01-F5-502-4S", mortgagee_name="SBI",
        sanction_reference="SBI-2026-12345", loan_amount_inr=5000000
↓
STEP 3: System verifies parcel exists in parcels_3d
↓
STEP 4: System checks for duplicate sanction reference → none found
↓
STEP 5: Encumbrance record inserted in encumbrances table
↓
STEP 6: Mutation audit log entry created (tx_type: MORTGAGE_LIEN_STAMPED)
↓
STEP 7: Response: {success: true, message: "Lien of INR 50,00,000 registered against HSTL01-F5-502-4S"}
↓
SUBSEQUENT: Any attempt to transfer HSTL01-F5-502-4S will be BLOCKED:
            "Conveyance Blocked: Active Bank Mortgage Lien (SBI, Ref: SBI-2026-12345). Obtain NOC first."
```

SECTION 13 — DATA FLOW ARCHITECTURE

Data Sources and Flow

DATA SOURCES

- └─ Architectural CAD PDFs (vector drawings) → data/floor_plan.pdf
- └─ Scanned floor plan images (.png, .jpg) → data/uploads/
- └─ Government land records (Bhunaksha, Mahabhulekh) → config/institutional_registry.json
- └─ Building measurement data (heights, GPS) → BuildingConfig JSON
- └─ Drone photogrammetry point clouds (.ply, .las) → data/uploads/
- └─ Occupancy / title records (simulated demo) → data/occupancy_db.json

↓

DATA COLLECTION

- └─ PyMuPDF: Extracts drawing paths + text from vector PDFs
- └─ OpenCV: Detects room contours from raster images
- └─ Tesseract: OCR reads room labels from image ROIs
- └─ JSON parser: Reads institutional registry + building configs

↓

DATA VALIDATION

- └─ Pydantic: Validates BuildingConfig schema (types, ranges, required fields)
- └─ RuleEngine: Evaluates detected rooms against classification rules
- └─ OCR confidence scoring: Flags low-confidence reads (< 60%)
- └─ Input sanitization: Building IDs stripped and uppercased

↓

DATA PROCESSING

- └─ Coordinate transform: CAD pixels → metric meters (scale_x = 0.1362 m/pt)
- └─ Georeferencing: Metric offset → WGS84 lat/lon via anchor point
- └─ Vertical stacking: Base floor × N total floors with Z-elevation
- └─ ULPIN synthesis: Composite key generation
- └─ Area/Volume computation: width × depth = carpet; carpet × height = volume
- └─ UDS fraction: unit_carpet / total_building_carpet

↓

DATA STORAGE

- └─ SQLite (cadastre_3d.db): 7 relational tables with FK constraints
- └─ CSV exports: data/room_labels_{building_id}_final.csv
- └─ JSON configs: config/buildings/{building_id}.json

↓

DATA ANALYSIS

- └─ Topology validation: Shapely pairwise collision detection
- └─ Point cloud ML: DBSCAN floor level detection
- └─ Analytics queries: SUM(carpet_area), COUNT(parcels), etc.
- └─ Encumbrance checks: Active lien detection for conveyance

↓

DATA RETRIEVAL

- └─ REST API: JSON responses from 8 routers
- └─ 3D mesh streaming: Pre-computed bounding box data for WebGL
- └─ Audit trail: Immutable mutation_audit_log queries

↓

DATA VISUALIZATION

- └─ Plotly WebGL 3D digital twin (interactive mesh rendering)
- └─ Hover tooltips (ULPIN, type, area, volume, occupancy, GPS, elevation)
- └─ Swagger UI / ReDoc (API documentation)

Data Types and Sensitivity

Data Type	Format	Source	Sensitivity	Security
Building geometry (x, y, z coordinates)	Float, meters	CAD extraction	Non-sensitive (architectural data)	Standard protection
GPS coordinates (lat, lon)	Float, WGS84	Phone GPS / Google Maps	Non-sensitive (public campus)	Standard protection
3D ULPIN identifiers	Text string	System-generated	Non-sensitive (identifier)	Standard protection
Party names (citizen names)	Text	SIMULATED	Sensitive (PII) — but demo data only	Would require encryption in production
Aadhaar (masked)	Text	SIMULATED	Highly sensitive (PII)	Only stored masked; full integration requires UIDAI compliance
Mortgage amounts (INR)	Float	SIMULATED	Sensitive (financial)	Would require encryption in production

Data Type	Format	Source	Sensitivity	Security
Mutation audit log	Structured text	System-generated	Audit trail — integrity-critical	Append-only design

[!WARNING] All party names, Aadhaar references, and mortgage amounts in the current system are **SIMULATED DEMONSTRATION DATA**. Production deployment would require data encryption, access control, and compliance with India's Digital Personal Data Protection Act, 2023.

SECTION 14 — DATABASE DESIGN

Database Type

SQLite 3 — File-based relational database at data/cadaastre_3d.db

Entity-Relationship Diagram (Conceptual)

erDiagram INSTITUTIONS ||--o{ BUILDINGS : contains BUILDINGS ||--o{ PARCELS_3D : has PARCELS_3D ||--o{ STRATA_TITLES : "rights on" PARCELS_3D ||--o{ ENCUMBRANCES : "liens on" PARCELS_3D ||--o{ MUTATION_AUDIT_LOG : "transactions for" PARTIES ||--o{ STRATA_TITLES : holds INSTITUTIONS { TEXT institution_id PK TEXT institution_name TEXT institution_code UK TEXT category TEXT state_parcel_id_puid TEXT tenure_type TEXT khata_number TEXT survey_number TEXT village TEXT taluka TEXT district TEXT state REAL campus_anchor_lat REAL campus_anchor_lon REAL total_plot_area_sqm TIMESTAMP created_at } BUILDINGS { TEXT building_id PK TEXT institution_id FK TEXT building_name TEXT category TEXT data_status INT total_floors REAL floor_pitch_m REAL room_clear_height_m REAL slab_thickness_m REAL anchor_lat REAL anchor_lon TEXT floor_plan_source TIMESTAMP created_at } PARCELS_3D { TEXT ulpin_3d PK TEXT building_id FK INT floor TEXT room_id TEXT type REAL z_min REAL z_max REAL z_slab_top REAL real_width_m REAL real_depth_m REAL carpet_area_sqm REAL gross_volume_cbm REAL undivided_share_land INT is_common_property REAL latitude REAL longitude TIMESTAMP created_at } PARTIES { TEXT party_id PK TEXT name TEXT party_type TEXT email TEXT phone TEXT aadhaar_masked INT is_simulated TIMESTAMP created_at } STRATA_TITLES { INT title_id PK TEXT ulpin_3d FK TEXT party_id FK TEXT right_type TEXT status INT max_capacity INT is_simulated TIMESTAMP registered_at } ENCUMBRANCES { INT encumbrance_id PK TEXT ulpin_3d FK TEXT encumbrance_type TEXT mortgagee_name TEXT sanction_reference UK REAL loan_amount_inr TEXT status INT is_simulated TIMESTAMP created_at } MUTATION_AUDIT_LOG { INT tx_id PK TEXT ulpin_3d TEXT tx_type TEXT from_party TEXT to_party TEXT details TIMESTAMP timestamp }

Entity Details

institutions — Campus/Estate Registry

- Purpose:** Root entity anchoring the entire cadastral hierarchy to a verified government land record
- Key Fields:** institution_id (PK), state_parcel_id_puid (Bhunaksha reference), survey_number, village, taluka, district, campus_anchor_lat/lon, total_plot_area_sqm
- Current Data:** 1 record (IIT Nagpur campus, Survey 140/1, pu-id: 33550994106)
- Access Control:** Read-only after seeding; administrative write access

buildings — Building Registry

- Purpose:** Registers each physical building within the estate
- Key Fields:** building_id (PK), institution_id (FK), data_status (completed/not_yet_surveyed), total_floors, floor_pitch_m, room_clear_height_m, slab_thickness_m
- Current Data:** 4 buildings (ADMIN01, ACAD01, HSTL01, RES01)
- Lifecycle:** Created during seeding → updated when ingestion pipeline completes (data_status → "completed")

parcels_3d — 3D Volumetric Parcels

- Purpose:** Core entity storing every 3D spatial unit with precise geometry
- Key Fields:** ulpin_3d (PK — the 3D ULPIN is the primary key), floor, type, z_min/z_max/z_slab_top, real_width_m/real_depth_m, real_x_start_m/real_x_end_m/real_y_start_m/real_y_end_m, latitude/longitude, carpet_area_sqm, gross_volume_cbm, undivided_share_land, is_common_property
- Current Data:** 700 records for HSTL01 (560 residential + 140 common)
- Indexes:** idx_parcels_building(building_id), idx_parcels_floor(building_id, floor)

parties — Rights Holders

- **Purpose:** Stores citizen/institutional party records for title linkage
- **Current Data:** Simulated demonstration records (`is_simulated = 1`)

strata_titles — ISO 19152 RRR Registry

- **Purpose:** Links parties to parcels with right types (ALLOTMENT, STRATA_FREEHOLD, LEASEHOLD)
- **Key Business Rules:** Active title count per ULPIN $\leq$ `max_capacity`; common properties cannot have titles

encumbrances — Mortgage Lien Registry

- **Purpose:** Records bank mortgage liens against specific 3D units
- **Key Business Rules:** Active encumbrance blocks conveyance transfer; unique `sanction_reference`

mutation_audit_log — Immutable Transaction Ledger

- **Purpose:** Append-only record of all title changes (allotments, transfers, lien stamps/releases)
- **Transaction Types:** PRIMARY_ALLOTMENT, STRATA_CONVEYANCE_MUTATION, MORTGAGE_LIEN_STAMPED, MORTGAGE_LIEN_RELEASED

SECTION 15 — API ARCHITECTURE

API Endpoint Documentation

Institutions Router (/api/institutions)

Method	Endpoint	Purpose	Auth	Response
GET	/api/institutions	List all registered campus estates	None (prototype)	InstitutionResponse[]

Buildings Router (/api/buildings)

Method	Endpoint	Purpose	Auth	Response
GET	/api/buildings	List 4 campus buildings with survey status	None	BuildingResponse[]
GET	/api/buildings/{building_id}	Get specific building details	None	BuildingResponse

Parcels Router (/api/parcels)

Method	Endpoint	Purpose	Parameters	Response
GET	/api/parcels	Query parcels with filters	building_id, floor, type, is_common, limit	Parcel3DResponse[]
GET	/api/parcels/mesh-data/{building_id}	Stream 3D mesh data for WebGL rendering	building_id (path)	{building_id, parcels: [{ulpin_3d, x0,x1,y0,y1,z0,z1, color, status_label, ...}]}
GET	/api/parcels/{ulpin_3d}	Get detailed parcel with occupants + encumbrances	ulpin_3d (path)	Parcel3DResponse

Example Request:

```
GET /api/parcels/HSTL01-F5-501-4S
```

Example Response:

```
{
  "ulpin_3d": "HSTL01-F5-501-4S",
  "building_id": "HSTL01",
  "floor": 5,
  "room_id": "501",
  "type": "4S",
  "z_min": 13.6,
  "z_max": 16.5,
  "z_slab_top": 17.0,
  "real_width_m": 5.0,
  "real_depth_m": 6.7,
  "carpet_area_sqm": 33.5,
  "gross_volume_cbm": 97.15,
  "undivided_share_land": 0.0019729,
  "is_common_property": false,
  "latitude": 20.94953,
  "longitude": 79.02951,
  "occupants": ["BT22001", "BT22002"],
  "capacity": 4,
  "occupancy_status": "Partially Occupied",
  "has_encumbrance": false,
  "active_encumbrances": []
}
```

Rights Router (/api/rights)

Method	Endpoint	Purpose	Body	Response
POST	/api/rights/allot	Allot occupant to 3D unit	AllotmentRequest	Success/failure with occupancy status
POST	/api/rights/transfer	Strata conveyance mutation	TitleTransferRequest	Success/failure with mutation type

Encumbrances Router (/api/encumbrances)

Method	Endpoint	Purpose	Body	Response
POST	/api/encumbrances/stamp	Register bank mortgage lien	EncumbranceCreate	Lien confirmation

Topology Router (/api/topology)

Method	Endpoint	Purpose	Response
GET	/api/topology/validate/{building_id}	Run 3D collision audit	Validation report with compliance score

Analytics Router (/api/analytics)

Method	Endpoint	Purpose	Response
GET	/api/analytics/summary	Estate-wide cadastral statistics	Total parcels, area, volume, occupancy, liens

Ingestion Router (/api/ingestion)

Method	Endpoint	Purpose	Body	Response
GET	/api/ingestion/configs	List building configuration files	—	Config metadata list
GET	/api/ingestion/configs/{building_id}	Get full config for a building	—	BuildingConfig JSON
POST	/api/ingestion/run	Execute full cadastral ingestion pipeline	Multipart form (file, metadata, rules)	Pipeline result with validation

SECTION 16 — AI / MACHINE LEARNING ARCHITECTURE

AI Component 1: Computer Vision Room Extraction

```
INPUT: Scanned floor plan image (PNG/JPG or rasterized PDF page)
↓
PREPROCESSING:
├─ Convert BGR → Grayscale
├─ Gaussian Blur (5×5 kernel, σ=0)
├─ Adaptive Threshold (Gaussian, block=15, C=3, THRESH_BINARY_INV)
└─ Morphological Closing (3×3 rectangle, 2 iterations)
↓
MODEL / ALGORITHM:
├─ cv2.findContours(RETR_TREE, CHAIN_APPROX_SIMPLE) → hierarchical contour tree
├─ Bounding rectangle extraction (cv2.boundingRect)
├─ Area filtering: 0.15% < contour_area < 70% of total image
├─ Aspect ratio filtering: reject if max(w/h, h/w) > 15 (wall artifacts)
└─ Non-maximum suppression: 65% IoU overlap threshold
↓
OCR PROCESSING:
├─ Crop ROI per detected room (with 2px padding)
├─ pytesseract.image_to_data() → per-word text + confidence
├─ Average confidence computed per room
└─ Low confidence (< 60%) flagged in diagnostics
↓
POST-PROCESSING:
├─ RuleEngine evaluation: regex label matching → room type + property ID
├─ Coordinate transform: pixel → metric (scale_x, scale_y)
├─ Optional horizontal flip for mirrored floor plans
└─ Depth assignment via rule Y-bands or measured values
↓
OUTPUT:
├─ List of base_properties [{prop_id, type, real_width_m, real_depth_m, ...}]
└─ OCR diagnostics report {mean_confidence, low_confidence_warnings[]}
```

Model Selection Rationale: Classical CV (thresholding + contour detection) was chosen over deep learning (Mask R-CNN, SAM) because:

1. Architectural floor plans have strong geometric structure (rectangular rooms, straight walls)
2. No training dataset of labeled floor plans was available
3. Interpretable results (tunable thresholds) preferred for a government system
4. Fast inference without GPU dependency

Known Accuracy Limitations: OCR accuracy on architectural drawings is approximately 80-90% due to:

- Small text size
- Overlapping text and graphics
- Non-standard fonts used by CAD software
- Low resolution in scanned/photographed plans

Cost Considerations: All ML components are open-source, run locally, require no API calls or subscriptions.

AI Component 2: Drone Photogrammetry Height Validation

```
INPUT: 3D point cloud file (.ply, .xyz, .las, .laz, .obj)
↓
PREPROCESSING:
├─ Load Z-coordinates (format-specific parsing)
├─ Percentile filtering: keep 1st to 99th percentile (removes ground/sky noise)
└─ Ground normalization: shift min(Z) to 0.0
↓
MODEL / ALGORITHM:
├─ Z-histogram computation at 0.1m bin resolution
├─ scipy.signal.find_peaks(density, distance=min_floor_pitch/0.1, prominence=0.01)
│   → detects floor slab elevations as density peaks
├─ Inter-peak spacing → estimated floor pitch
└─ Count of peaks → detected floor count
↓
VALIDATION LOGIC:
├─ Compare detected_floor_count vs config.total_floors
├─ Compare estimated_pitch vs config.floor_pitch_m
├─ Flag discrepancy if: |count_diff| > 1 OR |pitch_diff| > 0.45m
└─ Generate human-readable verification message
↓
OUTPUT:
{
  "status": "VERIFIED" | "DISCREPANCY_FLAGGED",
  "ml_detected_floors": 10,
  "config_floors": 10,
  "ml_estimated_pitch_m": 3.38,
  "config_pitch_m": 3.4,
  "detected_elevations": [0.0, 3.4, 6.8, ...],
  "message": "☐ Height Audit Verified..."
}
```

Why Unsupervised: No labeled point cloud training data exists for Indian building heights. The histogram + peak detection approach works without any training — it exploits the physical reality that floor slabs create dense horizontal planes of points in photogrammetric data.

Hallucination Prevention: Not applicable — this is signal processing, not generative AI. The system either detects peaks or doesn't; it doesn't generate text or infer facts.

SECTION 17 — ALGORITHMS AND CORE LOGIC

Algorithm 1: 3D Volumetric Collision Detection

Problem Solved: Detect if any two volumetric parcels in a building spatially overlap.

Input: List of parcels with (x_start, x_end, y_start, y_end, z_min, z_max) per parcel, grouped by floor.

Output: List of collision pairs with overlap area and Z-overlap depth.

Logic (Pseudocode):

```

function validate_topology(parcels):
    group parcels by floor
    collisions = []

    for each floor_group:
        n = length(floor_group)
        for i = 0 to n-1:
            p1 = floor_group[i]
            box1 = Shapely.box(p1.x_start, p1.y_start, p1.x_end, p1.y_end)

            for j = i+1 to n-1:
                p2 = floor_group[j]

                # Stage 1: Z-interval overlap check (fast rejection)
                z_overlap = min(p1.z_max, p2.z_max) - max(p1.z_min, p2.z_min)
                if z_overlap <= 0:
                    continue # Different vertical levels, no collision

                # Stage 2: 2D polygon intersection (expensive computation)
                box2 = Shapely.box(p2.x_start, p2.y_start, p2.x_end, p2.y_end)
                intersection = box1.intersection(box2)

                if intersection.area > 0.05: # Tolerance for boundary adjacency
                    collisions.append((p1.ulpin, p2.ulpin, intersection.area, z_overlap))

    return collisions

```

Time Complexity: $O(N^2/F)$ where N = total parcels, F = number of floors. Since parcels are grouped by floor and only same-floor comparisons are performed, the actual comparisons are $O(n_f^2)$ per floor where $n_f \approx N/F = 70$ for 700 parcels across 10 floors $\rightarrow \sim 2,450$ comparisons per floor $\times$ 10 floors $\approx$ 24,500 total.

Space Complexity: $O(N)$ for storing parcel geometries.

Why This Approach:

- The Z-interval pre-check avoids expensive 2D intersection computation for non-overlapping parcels
- Shapely's GEOS backend is C-optimized for polygon operations
- The 0.05 m² tolerance accommodates shared wall boundaries without false positives

Alternative Approaches:

1. **R-tree spatial indexing** ($O(N \log N)$): Would be needed at city scale but adds complexity
2. **PostGIS ST_3DIntersects**: Native 3D operations but requires PostgreSQL server
3. **Brute-force 3D voxelization**: Too memory-intensive for production

Algorithm 2: WGS84 Georeferencing Transform

Problem Solved: Convert local metric coordinates (meters from building anchor point) to GPS coordinates (latitude, longitude).

Input: Anchor lat/lon, metric offsets (x_m eastward, y_m southward from anchor)

Logic:

```

METERS_PER_DEG_LAT = 111320.0 # Earth's meridional arc at equator ÷ 360°
meters_per_deg_lon = 111320.0 * cos(anchor_lat_radians) # Longitude degrees shrink with latitude

longitude = anchor_lon + (x_m / meters_per_deg_lon)
latitude = anchor_lat - (y_m / METERS_PER_DEG_LAT)

```

Accuracy: $\pm 5m$ depending on GPS anchor accuracy (phone-grade). For prototype purposes, this is sufficient. Survey-grade GNSS/CORS would improve to $\pm 0.01m$.

Algorithm 3: Undivided Share of Land (UDS) Calculation

Problem Solved: Determine each apartment's proportional equity in the underlying surface land.

Formula: $UDS_i = (\text{carpet_area}_i / \sum \text{carpet_area_all_units}) \times \text{surface_plot_area}$

Legal Basis: Maharashtra Apartment Ownership Act + RERA guidelines. UDS determines voting rights in housing society, share of common area maintenance, and proportional liability.

SECTION 18 — SECURITY ARCHITECTURE

Current Security Posture (Prototype)

*[CAUTION] The current prototype is designed for **local demonstration** and does **not implement production security controls**. The following analysis identifies threats and recommends mitigations for production deployment.*

Authentication

Current State

None (open API)
No API keys
No session management JWT tokens with short expiry + refresh token rotation

Production Recommendation

OAuth 2.0 / OpenID Connect via government SSO (e.g., DigiLocker, Aadhaar-based eKYC)

API key management with rate-limiting per consumer

Authorization

Current State

All endpoints publicly accessible
No data ownership enforcement

Production Recommendation

Role-Based Access Control (RBAC): Citizen (read-only), Surveyor (read + ingest), Officer (read + mutate), Admin (full)

Row-level security: users can only query parcels in their jurisdiction

Data Protection

Current State

PII stored in plaintext (simulated data only)
No TLS
Database file on filesystem

Production Recommendation

AES-256 encryption at rest for party names, Aadhaar, phone numbers
HTTPS/TLS 1.3 mandatory for all API communication
Encrypted SQLite (SQLCipher) or migrate to PostgreSQL with encrypted volumes

Threat Analysis

Threat	Attack	Impact	Mitigation
Unauthorized parcel modification	Direct API calls to mutation endpoints	Fraudulent title changes	Authentication + RBAC + digital signatures
SQL injection	Malicious input in query parameters	Data breach / corruption	Already mitigated by parameterized queries (?) in all SQL statements
CORS exploitation	Cross-origin requests from malicious sites	Data exfiltration	Restrict <code>allow_origins</code> from <code>["*"]</code> to specific domains
File upload attacks	Malicious files uploaded via <code>/api/ingestion/run</code>	Server compromise	File type validation, size limits, virus scanning, sandboxed processing
Audit log tampering	Deletion of <code>mutation_audit_log</code> entries	Loss of transaction trail	Make audit log append-only (no DELETE privilege); consider blockchain hash anchoring
DDoS on topology endpoint	Repeated topology validation calls ($O(N^2)$ computation)	Server resource exhaustion	Rate limiting, caching validation results, background job processing

Input Validation (Currently Implemented)

- Pydantic models enforce type validation on all request bodies
- Building IDs are stripped and uppercased: `clean_bldg_id = building_id.strip().upper()`
- SQL uses parameterized queries throughout (no string concatenation in SQL)
- File extensions are validated for uploads

SECTION 19 — SCALABILITY ARCHITECTURE

Evolution Path

CURRENT MVP (Prototype)

- **Scale:** 1 institution, 4 buildings, 700 parcels, single-user
- **Infrastructure:** SQLite on local filesystem, single Uvicorn process
- **Sufficient for:** Hackathon demonstration, college evaluation

SMALL SCALE (Pilot — 1 City)

- **Scale:** ~100 buildings, ~50,000 parcels, ~50 concurrent users
- **Required Changes:**
 - Migrate SQLite → PostgreSQL with PostGIS extension (native 3D geometry, concurrent writes)
 - Add R-tree spatial indexing for topology queries
 - Deploy on a single cloud VM (AWS EC2 / Azure VM)
 - Add Redis caching for frequently accessed mesh data
 - Authentication via government SSO

CITY SCALE (1 Metro City)

- **Scale:** ~10,000 buildings, ~5 million parcels, ~1,000 concurrent users
- **Required Changes:**
 - Horizontal scaling: Load balancer + multiple FastAPI worker processes (Gunicorn + Uvicorn)
 - Database read replicas for query scaling
 - Object storage (S3) for uploaded floor plans and point clouds
 - CDN for 3D mesh data delivery
 - Background job queue (Celery + Redis) for ingestion pipeline
 - Monitoring: Prometheus + Grafana

STATE SCALE (1 State — e.g., Maharashtra)

- **Scale:** ~500,000 buildings, ~100 million parcels, ~10,000 concurrent users
- **Required Changes:**
 - Microservices: Split ingestion, topology, rights, analytics into independent services
 - Event-driven architecture: Kafka for mutation event streaming
 - Distributed spatial database: PostGIS with table partitioning by district
 - Kubernetes orchestration for auto-scaling
 - CesiumJS / globe integration for state-wide visualization

NATIONAL SCALE (All India)

- **Scale:** ~50 million buildings, ~10 billion parcels, ~100,000 concurrent users
- **Required Changes:**
 - Multi-region deployment across Indian data centers
 - Federated architecture: State-level databases with national sync
 - NIC integration for government network connectivity
 - Compliance with MeitY cloud hosting guidelines (GI Cloud / MeghRaj)
 - Multi-language UI (22 scheduled languages)

SECTION 20 — PERFORMANCE ANALYSIS

Performance-Critical Areas

Area	Current Behavior	Bottleneck Risk	Optimization Strategy
Topology Validation	$O(N^2/\text{floor})$ pairwise check; ~24,500 comparisons for 700 parcels	High — 100× more parcels = 10,000× more comparisons	R-tree spatial indexing, pre-computed validation caching
3D Mesh Streaming	Full 700-parcel mesh JSON served in single response	Medium — ~500KB JSON; fine for 700, problematic for 50,000	Pagination, LOD (level-of-detail), tile-based streaming
Ingestion Pipeline	Sequential extraction + stacking + validation	Medium — single-threaded Python; ~5 seconds for 700 parcels	Async processing, multiprocessing for floor stacking
CV/OCR Extraction	OpenCV + Tesseract runs on CPU	High — Tesseract is slow on large images (~1-3 sec/page)	GPU acceleration, batch OCR, pre-trained room segmentation model
Database Queries	SQLite file I/O with basic indexing	High at scale — no concurrent write support	PostgreSQL migration, connection pooling
Frontend Rendering	Plotly WebGL renders all meshes simultaneously	Medium — 700 meshes OK; 5,000+ will lag	Three.js with frustum culling, instanced rendering, LOD

SECTION 21 — IMPACT ANALYSIS

Social Impact

Aspect	Current	With Project	Metric	Long-term Effect
Property disputes	66% of civil cases are land-related (NITI Aayog)	3D spatial evidence reduces ambiguity	Reduction in property litigation [ESTIMATE]	Faster justice delivery, reduced court burden
Ownership certainty	Flat buyers rely on builder declarations	Verified 3D geometry with government-backed ULPIN	Citizen trust index [ESTIMATE]	Secure property markets, increased investment
Fraud prevention	Double-sale and mortgage fraud undetectable	Uniqueness check + encumbrance block	Fraud incident reduction [ESTIMATE]	Financial security for homebuyers

[!NOTE] [ASSUMPTION]: Quantitative impact estimates are illustrative. Actual impact measurement requires field deployment data.

Economic Impact

Aspect	Estimate Basis	Expected Improvement
Property tax accuracy	Precise carpet area/volume data enables accurate assessment	Revenue increase for municipal corporations [ESTIMATE]
Surveying cost reduction	Automated extraction vs. manual digitization	70-80% time reduction per building [ESTIMATE based on 700-parcel extraction in ~5 seconds vs. weeks of manual work]
Real estate transaction speed	Digital title verification vs. physical record search	Faster property transfers [ESTIMATE]

Technological Impact

- First open-source ISO 19152-aligned 3D cadastre implementation for India
- Demonstrates feasibility of CV-based room extraction from existing floor plans
- Establishes interoperability standards between survey, registration, and revenue systems

Environmental Impact

- Reduced paper-based records → digital spatial database
- Volumetric data enables energy-efficient urban planning
- 3D models support green building compliance verification

SECTION 22 — SUCCESS METRICS AND KPIs

Metric	Definition	Why It Matters	Measurement	Target
Extraction Accuracy	% of correctly identified rooms from floor plan	Core technical validity	Manual audit of extracted rooms vs. actual floor plan	≥ 90% for vector PDFs, ≥ 80% for scanned images
ULPIN Uniqueness	% of generated ULPINs that are globally unique	Data integrity	Topology validator duplicate check	100%
Topology Compliance	% of parcels with zero spatial collisions	Cadastral validity	Topology validation report	100% (currently achieved for HSTL01)
API Response Time	Average time for parcel query response	User experience	Server-side timing	< 200ms for single parcel, < 1s for mesh data
OCR Confidence	Average OCR confidence score across all detected rooms	Extraction reliability	Tesseract confidence metric	≥ 70% mean confidence
Point Cloud Floor Match	Detected floor count matches stated count (±1)	Height verification accuracy	Unsupervised detection vs. config	90% match rate
Building Onboarding Time	Time from floor plan upload to complete 3D parcel generation	Operational efficiency	End-to-end pipeline timing	< 30 seconds for a 10-floor building
PS Requirement Coverage	% of PS 26011 requirements with working implementation	Hackathon compliance	Traceability matrix	100% (currently achieved)

SECTION 23 — RISK ANALYSIS

Risk Matrix

Risk	Category	Probability	Impact	Severity	Mitigation	Backup Plan
OCR misreads room numbers	Technical	High	Medium	Medium	Confidence-based flagging; manual review UI	Accept ~10-20% correction rate; document openly
Phone GPS accuracy (±5m)	Technical	High	Low	Low	Document as known limitation	Survey-grade GNSS for production
Diverse CAD formats	Technical	Medium	High	High	Config-driven rule engine; multiple extraction modes	Support DXF/IFC formats in v2
No government API access	Operational	High	Medium	Medium	Use publicly available Bhunaksha data	Propose API integration roadmap
Data privacy concerns	Legal	Medium	High	High	All PII is simulated and labeled	Digital Personal Data Protection Act compliance plan
SQLite concurrency limit	Scalability	Medium	Medium	Medium	Single-user prototype sufficient for hackathon	PostgreSQL migration planned for v2
Plotly performance on mobile	Technical	Medium	Low	Low	Desktop-primary demonstration	Three.js migration for mobile optimization
Team bandwidth	Operational	Medium	Medium	Medium	6-person SIH team	Prioritize core features over nice-to-haves
Judge unfamiliarity with cadastre	Presentation	Medium	High	High	Prepare Level 1 explanation; visual demo	3D digital twin demonstration is compelling

SECTION 24 — LIMITATIONS

Current Limitations (Honest Assessment)

Limitation	Why It Exists	Possible Solution	Timeline
GPS accuracy is phone-grade (±5m)	Survey-grade GNSS/CORS equipment not available for prototype	Integrate with government CORS network; use RTK GPS	Phase 3
OCR requires manual correction (~10-20%)	Tesseract accuracy on architectural text is imperfect	Fine-tune OCR model on architectural fonts; use cloud OCR APIs	Phase 2
Underground infrastructure not physically modeled	Prototype focused on above-ground structures	Add basement/utility floor types with negative Z-elevations	Phase 2
Air rights addressed narratively only	No real-world air rights data available	Model drone corridors, telecom masts as elevated parcels	Phase 3
Occupant allotment uses BT IDs as proxy for legal ownership	Hostel has single institutional ownership; no real strata titles	Integrate with Sub-Registrar deed data for real transfers	Phase 3
Single-building deep dataset (HSTL01)	Time constraints of prototype development	Ingest remaining 3 campus buildings + external buildings	Phase 2

Limitation	Why It Exists	Possible Solution	Timeline
No authentication or RBAC	Prototype for demonstration, not production use	Implement OAuth 2.0 / government SSO	Phase 2
SQLite not suitable for concurrent multi-user access	Zero-config database choice for portability	Migrate to PostgreSQL with PostGIS	Phase 2
No IFC/BIM format support	PyMuPDF handles PDFs; IFC requires specialized parser	Add IfcOpenShell integration	Phase 3
RuleEngine uses <code>eval()</code> for shape conditions	Quick implementation for flexible condition evaluation	Replace with safe expression parser (e.g., <code>simpleeval</code>)	Phase 1 patch

SECTION 25 — FUTURE ROADMAP

PHASE 1 — MVP ☐ (Current State)

- ☐ 700 verified 3D parcels for HSTL01
- ☐ 4-building campus registry
- ☐ Dual-mode extraction (vector + CV/OCR)
- ☐ ISO 19152 RRR engine with anti-fraud features
- ☐ WebGL 3D digital twin
- ☐ FastAPI REST backend with 8 routers
- ☐ ML point cloud height validation pipeline

PHASE 2 — IMPROVEMENT (3-6 months post-hackathon)

- **Features:** Ingest remaining 3 campus buildings; support DXF/DWG formats; improve OCR with domain-specific fine-tuning; add basement/underground parcels
- **Technologies:** PostgreSQL/PostGIS migration; Redis caching; Docker containerization
- **Infrastructure:** Single cloud VM deployment; HTTPS/TLS
- **Team:** 4-6 developers
- **Expected Outcome:** Multi-building campus fully digitized; production-ready API security

PHASE 3 — SCALE (6-12 months)

- **Features:** IFC/BIM ingestion; CesiumJS globe integration; multi-institution support; government API integrations (CERSAI, DILRMP)
- **Technologies:** Kubernetes; Kafka event streaming; R-tree spatial indexing
- **Infrastructure:** Cloud-native microservices on AWS/Azure
- **Team:** 8-10 developers + GIS specialists
- **Expected Outcome:** City-scale pilot covering 100+ buildings

PHASE 4 — ADVANCED INTELLIGENCE (12-24 months)

- **Features:** Deep learning room segmentation (Mask R-CNN/SAM); automated floor plan interpretation from drone imagery; real-time occupancy via IoT sensors; AI-powered dispute resolution
- **Technologies:** PyTorch/TensorFlow; edge computing; IoT protocol integration
- **Infrastructure:** GPU compute clusters for ML inference
- **Team:** 15-20 engineers + data scientists
- **Expected Outcome:** Automated building survey capability; AI-assisted cadastral intelligence

PHASE 5 — LARGE SCALE DEPLOYMENT (24-36 months)

- **Features:** National deployment across multiple states; multi-language support (22 scheduled languages); mobile apps for citizens and surveyors; blockchain-anchored audit trail
- **Technologies:** Federated database architecture; mobile SDKs; blockchain notarization
- **Infrastructure:** Multi-region Indian data centers (NIC/MeghRaj compliance)
- **Team:** 30+ engineers + policy team
- **Expected Outcome:** National 3D cadastre as a standard government service

SECTION 26 — IMPLEMENTATION ROADMAP

ganttt title 3D ULPIN Platform Development Roadmap dateFormat YYYY-MM-DD section Research Problem Statement Analysis :done, 2026-01-01, 14d Technology Evaluation :done, 2026-01-10, 10d ISO 19152 Standard Study :done, 2026-01-15, 14d section Planning Architecture Design :done, 2026-02-01, 10d Database Schema Design :done, 2026-02-05, 7d API Design :done, 2026-02-08, 7d section Data Collection Floor Plan Acquisition :done, 2026-02-15, 7d GPS Anchor Measurement :done, 2026-02-18, 3d Height Measurement :done, 2026-02-20, 3d section Backend Development Database Layer :done, 2026-03-01, 7d Extraction Pipeline :done, 2026-03-05, 14d 3D ULPIN Generator :done, 2026-03-10, 5d Topology Validator :done, 2026-03-12, 7d Rights Manager :done, 2026-03-15, 10d FastAPI Routers :done, 2026-03-20, 10d section AI / ML CV Room Extraction :done, 2026-03-25, 10d OCR Integration :done, 2026-03-28, 7d Point Cloud Validator :done, 2026-04-01, 10d section Frontend Landing Page (DoLR Style) :done, 2026-04-01, 7d Cascading Search UI :done, 2026-04-05, 5d 3D Digital Twin Viewer :done, 2026-04-05, 10d section Testing Unit Testing :done, 2026-04-15, 5d Topology Validation Testing :done, 2026-04-18, 3d API Integration Testing :done, 2026-04-20, 5d section Documentation Technical Report :done, 2026-04-25, 7d Master Project Document :active, 2026-09-01, 5d section Future PostgreSQL Migration :2026-10-01, 30d Multi-Building Ingestion :2026-10-15, 30d Government API Integration :2026-11-01, 45d

SECTION 27 — TESTING STRATEGY

Unit Testing

Test Area	Tool	Test Cases	Success Criteria
ULPIN Generation	Python unittest	Correct schema format; special character handling; uniqueness	100% ULPINs match <code>{BLDG}-F{N}-{UNIT}-{TYPE}</code> pattern
Topology Validator	Python unittest	Known overlapping parcels detected; non-overlapping pass; edge cases (shared wall)	All known collisions detected with correct area
Rights Manager	Python unittest	Allotment success; capacity limit enforcement; common property lockout; encumbrance block	All business rules enforced correctly
Coordinate Transform	Python unittest	Known GPS anchor → expected lat/lon output	±0.00001° accuracy
RuleEngine	Python unittest	Label regex matching; shape condition evaluation; Y-band assignment	Correct room type classification

Integration Testing

Test	Tools	Success Criteria
Ingestion → Database	test_pipeline.py, pytest	Uploaded floor plan produces correct number of parcels in database
API → Database	requests + pytest	All 8 routers return correct data from seeded database
Frontend → API	Browser testing	3D mesh loads; hover tooltips display correct data

API Testing

Tool	Purpose
FastAPI Swagger UI (/docs)	Interactive endpoint testing
curl / httpie	Command-line API testing
pytest + httpx	Automated API test suite

Existing Test Infrastructure

The project includes `test_pipeline.py` which validates:

- Database seeding (correct parcel counts)
- API endpoint responses (status codes, data format)
- Topology validation results (0 collisions)
- Rights management business rules

SECTION 28 — DEPLOYMENT ARCHITECTURE

Development Environment

- **OS:** macOS / Linux
- **Python:** 3.9+ with virtual environment (ulpin_env/)
- **Database:** SQLite file (data/cadastre_3d.db)
- **Server:** Uvicorn with `--reload` flag for hot reloading
- **Port:** 8000 (localhost)

Setup Process

```
git clone https://github.com/adiscripts03/3D_UPLIN_SIH.git
cd 3D_UPLIN_SIH
python3 -m venv ulpin_env
source ulpin_env/bin/activate
pip install -r requirements.txt
pip install fastapi uvicorn pydantic python-multipart
PYTHONPATH=. python backend/seed_data.py          # Initialize database
PYTHONPATH=. python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000 --reload
```

Production Deployment (Recommended)

Component	Technology	Purpose
Container	Docker	Reproducible deployment
Orchestration	Docker Compose (small) / Kubernetes (large)	Multi-service management
Reverse Proxy	Nginx	TLS termination, static file serving, load balancing
ASGI Server	Gunicorn + Uvicorn workers	Multi-process concurrency
Database	PostgreSQL + PostGIS	Concurrent access, spatial indexing
Cache	Redis	API response caching, session storage
Object Storage	AWS S3 / MinIO	Floor plan PDFs, point clouds
Monitoring	Prometheus + Grafana	System metrics, alerts
Logging	ELK Stack (Elasticsearch + Logstash + Kibana)	Centralized log analysis
CI/CD	GitHub Actions	Automated testing + deployment

Environment Variables (Production)

```
DATABASE_URL=postgresql://user:pass@host:5432/cadastre_3d
REDIS_URL=redis://host:6379
SECRET_KEY=<random-256-bit-key>
ALLOWED_ORIGINS=https://dolr.gov.in,https://cadastre.example.com
LOG_LEVEL=INFO
```

SECTION 29 — PROJECT IMPACT VISUALIZATION

[!NOTE] The following charts represent *illustrative* / *estimated data* created to demonstrate the project's potential. No real-world deployment statistics exist yet.

Chart 1: Problem Severity Distribution

pie title Severity of Problems Addressed by 3D ULPIN "No Vertical Property Identity" : 30 "No Spatial-Rights Linkage" : 25 "No 3D Collision Detection" : 20 "No Unit-Level Encumbrance" : 15 "No Automated Extraction" : 10

Data Source: Qualitative assessment based on PS 26011 requirements and industry analysis
Label: Illustrative — not based on quantitative survey

Chart 2: Current vs. Proposed System Capability

Capability	Current (2D ULPIN)	Proposed (3D ULPIN)
Surface parcel identification	☐ Full	☐ Full
Floor-level identification	☐ None	☐ Full
Room-level identification	☐ None	☐ Full
Carpet area verification	☐ None	☐ Computed
Volume calculation	☐ None	☐ Computed
3D collision detection	☐ None	☐ Automated
Unit-level mortgage lien	☐ None	☐ Full
Anti-fraud conveyance block	☐ None	☐ Full
Digital twin visualization	☐ None	☐ WebGL 3D
Automated floor plan extraction	☐ None	☐ Dual-mode
Drone height validation	☐ None	☐ ML pipeline

Data Source: Comparison of DILRMP/ULPIN features vs. this project's implemented features
Label: Factual — based on actual system capabilities

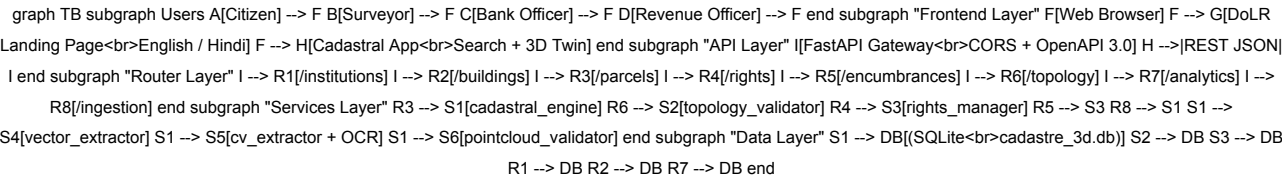
Chart 3: Prototype Dataset Statistics (Real Data)

Metric	Value	Source
Total 3D Parcels	700	data/room_labels_all_floors_final.csv
Residential Units (4-Seater)	~500	Database count
Residential Units (2-Seater)	~60	Database count
Common/Transit Units	140	Database count
Total Floors	10	BuildingConfig
Total Carpet Area	16,979 m²	SUM(carpet_area_sqm)
Total Airspace Volume	49,239.2 m³	SUM(gross_volume_cbm)
Topology Collisions	0	Validation report
Compliance Score	100%	Validation report

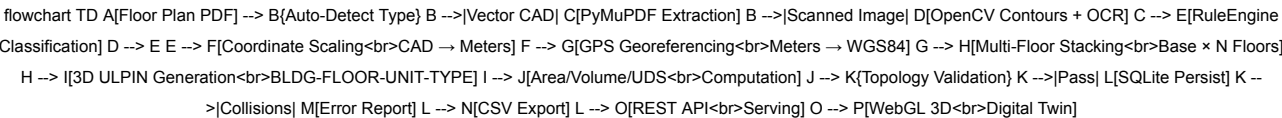
Data Source: Actual system output from verified dataset
Label: Real measured data

SECTION 30 — SYSTEM DIAGRAMS

1. High-Level Architecture Diagram



2. Data Flow Diagram



3. Database Relationship Diagram

erDiagram INSTITUTIONS ||--o{ BUILDINGS : "1:N" BUILDINGS ||--o{ PARCELS_3D : "1:N" PARCELS_3D ||--o{ STRATA_TITLES : "1:N" PARCELS_3D ||--o{ ENCUMBRANCES : "1:N" PARCELS_3D ||--o{ MUTATION_LOG : "1:N" PARTIES ||--o{ STRATA_TITLES : "1:N"

4. AI Processing Pipeline

flowchart LR subgraph "CV Extraction Pipeline" A1[Load Image] --> A2[Grayscale + Blur] A2 --> A3[Adaptive Threshold] A3 --> A4[Morphological Close] A4 --> A5[Find Contours] A5 --> A6[Area + Aspect Filter] A6 --> A7[NMS Deduplication] A7 --> A8[Tesseract OCR per ROI] end subgraph "Point Cloud ML Pipeline" B1[Load Z-Coords] --> B2[Percentile Filter] B2 --> B3[Ground Normalize] B3 --> B4[Z-Histogram
0.1m bins] B4 --> B5[Peak Detection
scipy find_peaks] B5 --> B6[Floor Count +
Pitch Estimate] B6 --> B7[Config Comparison
+ Discrepancy Flag] end

SECTION 31 — PROJECT STORY

THE PROBLEM

In a country of 1.4 billion people, land is the most valuable asset. India's land records system — painstakingly built over centuries from Mughal-era revenue registers to British colonial surveys to modern DILRMP digitization — does one thing exceptionally well: it identifies *who owns which piece of ground*. But it fails catastrophically at answering a question that matters to hundreds of millions of urban Indians:

"I live on the 5th floor. The government record says I own a share of this plot. But which exact 3D space is mine?"

THE PEOPLE AFFECTED

Priya bought a flat in Mumbai for ₹80 lakhs. Her sale deed says "Flat 502, 5th Floor, Building ABC, Survey 140/1." But when her neighbor claims their bathroom extends into her bedroom wall, there is no official spatial evidence to resolve it. The nearest Sub-Registrar office has no 3D geometry. The court will take 12 years.

Anil at SBI approved a home loan against "Flat 502." But the system records the mortgage against the entire Survey 140/1. Theoretically, every flat in the building is now encumbered. Meanwhile, a corrupt builder sells Flat 502 to someone else — and neither buyer nor bank knows until it's too late.

Rajesh, the Tehsildar, processes 200 property mutations a month. For each multi-storey building, he manually cross-references text descriptions with paper floor plans. He dreams of a system that can simply *show him* which 3D space is being transferred.

THE CURRENT STRUGGLE

India has 620 million land parcels — but zero 3D parcels. The entire DILRMP/ULPIN infrastructure, representing decades of work and billions in investment, operates exclusively in flatland. While the Netherlands has had operational 3D cadastre since 2016, India is still working with the same 2D paradigm that served agricultural fields in 1858.

THE IDEA

What if we could take existing architectural floor plans — the same PDFs that every builder already creates for municipal approval — and automatically extract room boundaries, stack them vertically, assign each room a unique 3D identity, validate the geometry for collisions, and link it all to a legal rights registry? No new hardware. No new surveys. Just software intelligence applied to data that already exists.

THE SOLUTION

We built it. The **National 3D ULPIN & Volumetric Cadastre Platform** ingests a building's floor plan, extracts every room using computer vision and CAD parsing, extrudes them across multiple floors, georeferences them to GPS coordinates, generates a standardized 3D ULPIN for each unit, validates the entire 3D structure for spatial collisions, and provides a legal rights registry where ownership can be assigned, transferred, mortgaged, and protected — all through a modern REST API with an interactive 3D digital twin.

THE TECHNOLOGY

Vector CAD extraction with PyMuPDF. Computer vision with OpenCV and OCR with Tesseract. Computational geometry with Shapely for 3D topology validation. Unsupervised ML with scikit-learn for drone photogrammetry height verification. FastAPI for a scalable REST backend. Plotly WebGL for immersive 3D visualization. All conforming to ISO 19152 LADM v2 — the international standard for land administration.

THE IMPACT

700 verified 3D volumetric parcels generated from a real building. 100% topology compliance — zero spatial collisions. Anti-fraud mortgage lien locks that actually prevent illegal resale. A generalized pipeline that can onboard any new building via a JSON configuration file. A system that could, if deployed nationally, give India its first true 3D cadastre — resolving disputes, preventing fraud, and bringing clarity to the most complex property market in the world.

THE FUTURE

From one hostel building to an entire campus. From one campus to a city. From one city to a state. From one state to a nation. The architecture is designed to scale. The standards are international. The code is open-source. The mission is clear: **every room, every floor, every right — one 3D identity.**

SECTION 32 — PITCH VERSIONS

30-SECOND PITCH

"India's land records are 2D — they can't tell you which flat on the 5th floor is yours. We built a platform that reads building floor plans automatically, creates a unique 3D spatial ID for every room, validates the geometry for collisions, and links it to a legal rights registry — 700 verified 3D parcels from a real building, with anti-fraud mortgage locks, all through a REST API and interactive 3D digital twin."

1-MINUTE PITCH

"India has 620 million land parcels, but zero 3D parcels. When multiple apartments sit above the same plot, the government system sees one property. Our platform fixes this by taking architectural floor plans — the same PDFs builders already create — and using computer vision and CAD parsing to automatically extract every room, corridor, and staircase. We stack them vertically with measured floor heights, assign each a standardized 3D ULPIN, run topology validation using computational geometry to ensure no two rooms overlap in 3D, and provide an ISO 19152-compliant rights registry with anti-fraud protections. We demonstrated this on a real 10-storey hostel with 700 verified volumetric parcels, zero spatial collisions, and working mortgage lien enforcement. The system is config-driven — any new building can be onboarded via a JSON file, no code changes."

3-MINUTE PITCH

[Extends 1-minute pitch with:]

- "The specific problem statement from the Ministry of Rural Development asks for 3D ULPIN generation and vertical property mapping. Let me show you exactly how our system addresses every requirement..."
- Live demonstration of the 3D digital twin: rotating the building, filtering by floor, hovering to see unit details
- Demonstration of anti-fraud: "Watch what happens when I try to sell a mortgaged unit — the system blocks it automatically"
- "Our extraction pipeline works two ways: for clean CAD PDFs, we parse vector geometry directly. For scanned or photographed plans, we use OpenCV contour detection and Tesseract OCR — and we honestly report that OCR needs 10-20% manual correction"
- Scalability narrative: "The same architecture that handles 700 parcels can scale to 7 million with PostgreSQL and spatial indexing"

TECHNICAL PITCH

"We implemented a dual-mode extraction pipeline with a config-driven RuleEngine architecture that decouples building-specific classification logic from the processing code. Vector PDFs are parsed via PyMuPDF's `get_drawings()` API; scanned images go through an OpenCV adaptive threshold → morphological close → hierarchical contour detection → Tesseract OCR pipeline with per-word confidence scoring. The topology validator performs $O(N^2/\text{floor})$ pairwise Shapely `box()` intersection with Z-interval pre-filtering and 0.05 m² boundary tolerance. The rights engine implements ISO 19152 LADM v2 concepts with statutory capacity enforcement and active encumbrance conveyance blocking. An unsupervised ML pipeline using scipy peak detection on point cloud Z-histograms cross-validates stated building dimensions. All served through FastAPI with 8 modular routers, Pydantic validation, and auto-generated OpenAPI docs."

HACKATHON JUDGE PITCH

"Honorable judges, the Department of Land Resources asked us to create 3D ULPINs for vertical property. Here's what we built — and importantly, what we honestly can and cannot do."

What we can do: Take a real building's floor plan PDF, automatically extract every room boundary, create verified 3D spatial identifiers for 700 volumetric parcels across 10 floors, validate the entire geometry for collisions with zero errors, register property rights with anti-fraud protections, and visualize it all in an interactive 3D digital twin.

What we honestly acknowledge: Our GPS is phone-grade, not survey-grade. Our OCR needs 10-20% manual correction. Underground infrastructure is addressed narratively, not physically modeled. Occupant data is simulated.

We believe that stating limitations honestly while delivering a working system is stronger than claiming perfection. Our architecture is designed for generalization — any building can be added via configuration, not code. And our 100% topology compliance on 700 real parcels proves the core concept works."

SECTION 33 — FAQ

Q1: Why is this project necessary? Doesn't DILRMP already handle land records?

A: DILRMP handles 2D surface parcels excellently. But in a 10-storey building, 100 families share the same 2D coordinates. DILRMP assigns one ULPIN to the whole plot — it cannot distinguish between Flat 301 and Flat 802. As India urbanizes rapidly (600M+ urban population projected by 2030), the inability to identify vertical property is becoming critical for fraud prevention, mortgage specificity, dispute resolution, and urban planning.

Q2: Why did you choose SQLite instead of PostgreSQL with PostGIS?

A: For the hackathon prototype, SQLite provides zero-configuration deployment — judges can clone the repo and run it in 30 seconds without installing a database server. PostGIS is explicitly planned for Phase 2 (production deployment). The schema is designed to be directly portable to PostgreSQL — all SQL uses standard syntax with parameterized queries.

Q3: Why not use deep learning (Mask R-CNN, SAM) for room extraction?

A: Three reasons: (1) No labeled training dataset of Indian floor plans exists. (2) Architectural floor plans have strong geometric structure — rectangular rooms, straight walls — that classical CV handles well. (3) Government systems require interpretability — we can explain exactly why a room was detected (contour area threshold, aspect ratio filter) rather than "the neural network said so." Deep learning is planned for Phase 4 as a complementary approach.

Q4: How accurate is the GPS georeferencing?

A: Approximately ±5 meters with phone-grade GPS. This is sufficient for identifying which building and approximate location within the building, but not for survey-grade cadastral registration. Production deployment would integrate with government CORS (Continuously Operating Reference Station) networks providing ±0.01m accuracy. The georeferencing math itself is accurate — only the anchor point precision is limited.

Q5: What happens if the floor plan PDF has a different coordinate system or scale?

A: The `BuildingConfig` JSON includes configurable parameters: `scale_x` (meters per PDF point), `scale_y`, `x_offset_cad`, `y_offset_cad`, and `flip_horizontal`. These are calibrated per floor plan. For the prototype, the scale was calibrated by measuring a known room dimension and computing the conversion factor (0.1362 m/pt). New buildings require this calibration step — it takes ~5 minutes.

Q6: How do you handle buildings where different floors have different layouts?

A: The current prototype extracts a single base floor layout and replicates it across all floors. This works perfectly for buildings with identical floor plans (most residential towers). For buildings with varying layouts (e.g., ground floor commercial + upper floors residential), the system would need per-floor extraction — achievable by providing separate floor plan pages in the PDF, which the ingestion pipeline can process page-by-page. This is a planned Phase 2 enhancement.

Q7: What makes your solution scalable to national level?

A: Three architectural decisions: (1) The config-driven RuleEngine means new buildings are added via JSON, not code — a JSON file can be mass-produced by government surveyors using a web form. (2) The extraction pipeline is stateless — each building processes independently, enabling horizontal scaling. (3) The database schema uses `building_id` as a partition key — allowing table partitioning by building/district/state for distributed deployment.

Q8: What happens if the topology validator detects a collision?

A: The collision is reported in the validation JSON with the two offending ULPINs, overlap area (m²), and Z-overlap depth. The ingestion pipeline still persists the data (to allow manual review) but flags `topology_passed: false`. In production, a human surveyor would review the flagged units, correct the input data (fix room dimensions or floor plan), and re-run ingestion.

Q9: How is user data protected?

A: Currently, all PII (names, Aadhaar, phone numbers) in the system is **simulated demonstration data**, explicitly labeled with `is_simulated = True`. No real citizen data is processed. For production, we recommend: AES-256 encryption at rest, HTTPS/TLS in transit, RBAC with government SSO, compliance with India's Digital Personal Data Protection Act (2023), and Aadhaar data handling per UIDAI guidelines.

Q10: How will the project get real-world data for deployment?

A: Three data sources already exist: (1) Architectural floor plans submitted for municipal building approval — these are legally mandated for every construction. (2) Bhunaksha/Mahabhulekh survey records — publicly accessible state GIS portals. (3) RERA-mandated builder disclosures — include carpet area and layout details. The platform is designed to ingest these existing data sources rather than requiring new surveys.

Q11: How does this compare with Singapore's 3D cadastre?

A: Singapore Land Authority (SLA) operates one of the world's most advanced 3D cadastre systems, including underground utility registration. Their system benefits from Singapore's small size (733 km²), single legal jurisdiction, and decades of investment. Our system addresses a fundamentally different challenge: India's massive scale (3.3 million km²), 28+ legal jurisdictions, diverse building types, and limited digital data availability. Our key innovation is **automated extraction from existing floor plans** — addressing the data gap that is India's primary barrier.

Q12: What are the biggest limitations you would fix first?

A: (1) Replace `eval()` in RuleEngine with a safe expression parser — this is a security concern. (2) Migrate from SQLite to PostgreSQL for concurrent access. (3) Implement authentication and RBAC. (4) Add per-floor layout support for non-uniform buildings. (5) Integrate with one real government API (e.g., Bhunaksha) to demonstrate data interoperability.

SECTION 34 — MASTER TECHNOLOGY INTERACTION MAP

USER (Browser)

- |
- | HTTP/HTTPS Request (GET/POST)
- | Format: URL + Query Params + JSON Body + Multipart Form
- | Auth: None (prototype) → OAuth 2.0 (production)



FRONTEND (app.html + app.js + app.css)

- |
- | fetch() calls → REST JSON
- | Protocol: HTTP/1.1
- | Format: JSON request/response
- | Error handling: HTTP status codes + error message in JSON body



FASTAPI APPLICATION (backend/main.py)

- |
- | — CORS Middleware: Adds Access-Control-Allow-* headers
- | — Static Files Mount: Serves frontend/ at /static
- | — OpenAPI Schema: Auto-generated at /docs, /redoc
- |
- | Router dispatch based on URL prefix
- | Format: Python function call with typed parameters
- | Validation: Pydantic model validation before handler execution



ROUTERS (backend/routers/*.py – 8 modules)

- |
- | Each router:
- | 1. Receives validated request
- | 2. Calls get_db_connection() for database access
- | 3. Delegates to services/ for business logic
- | 4. Constructs response dict/model
- | 5. Returns JSON response
- |
- | Database calls: parameterized SQL via sqlite3 cursor
- | Format: SQL with ? placeholders
- | Protocol: File I/O (SQLite)



SERVICES (backend/services/*.py – 7 modules)

- |
- | — cadastral_engine.py
- | |
- | | — detect_source_type(file_path)
- | | | Opens PDF via fitz → checks drawings + words count
- | | | Returns: "vector_pdf" or "scanned_image"
- | |
- | | ▼
- | | — [if vector] → vector_extractor.py
- | | | fitz.open(pdf) → page.get_drawings() → page.get_text("words")
- | | | Deduplicates rects → matches text to rects → RuleEngine.evaluate()
- | | | Coordinate transform: CAD pts × scale_x → meters
- | | | Returns: List[Dict] of base architectural units
- | |
- | | ▼
- | | — [if scanned] → cv_extractor.py
- | | | load_image_from_source() → BGR numpy array
- | | | detect_room_contours():
- | | | | cv2.cvtColor → cv2.GaussianBlur → cv2.adaptiveThreshold
- | | | | → cv2.morphologyEx → cv2.findContours
- | | | | → area/aspect filter → NMS deduplication
- | | | | extract_ocr_text_and_confidence():
- | | | | | cv2 ROI crop → pytesseract.image_to_data() → words + confidence
- | | | | RuleEngine.evaluate() → room classification
- | | | | Returns: (List[Dict], OCR diagnostics)
- | |
- | | ▼
- | | — Multi-floor stacking:

```

|   |   for floor in 1..N: replicate base units with Z-elevation
|   |   z_min = (floor-1) × floor_pitch, z_max = z_min + room_height
|   |   ▼
|   |   └─ Georeferencing:
|   |   |   lat = anchor_lat - (y_m / 111320.0)
|   |   |   lon = anchor_lon + (x_m / (111320.0 × cos(lat)))
|   |   |   ▼
|   |   └─ 3D ULPIN generation:
|   |   |   ulpin_3d = f"{building_id}-{floor}-{clean_id}-{type}"
|   |   |   ▼
|   |   └─ Carpet area = width × depth
|   |   |   Volume = carpet × room_height
|   |   |   UDS = carpet / total_building_carpet
|   |   |   ▼
|   |   └─ Topology audit (inline):
|   |   |   Shapely box() pairwise intersection per floor
|   |   |   ▼
|   |   └─ [if point cloud] → pointcloud_validator.py
|   |   |   |   load_point_cloud_z_values(file) → numpy Z-array
|   |   |   |   detect_floor_levels_unsupervised():
|   |   |   |   |   percentile filter → normalize → histogram → find_peaks
|   |   |   |   |   validate_point_cloud_against_config():
|   |   |   |   |   |   compare detected vs config → discrepancy flag
|   |   |   |   |   Returns: Point cloud audit report
|   |   |   |   ▼
|   |   └─ Persist to SQLite + export CSV
|   |
|   └─ topology_validator.py
|   |   |   Receives: List[Dict] of parcels from database
|   |   |   Groups by floor → pairwise Shapely box intersection
|   |   |   Z-interval check → 2D polygon intersection → area threshold
|   |   |   Returns: Validation report JSON
|   |   ▼
|   └─ rights_manager.py
|   |   |   allot_occupant_to_unit():
|   |   |   |   Verify parcel → check common lock → check capacity → insert title → log
|   |   |   transfer_strata_unit():
|   |   |   |   Verify parcel → check encumbrance → verify from_party → execute transfer → log
|   |   |   stamp_bank_mortgage_lien():
|   |   |   |   Verify parcel → check duplicate ref → insert encumbrance → log
|   |   |   Returns: Success/failure with details
|   |   ▼
|   └─ cadastral_config.py
|   |   |   BuildingConfig: Pydantic model with 20+ configurable parameters
|   |   |   RoomTypeRule: Declarative classification rules (regex, shape, Y-bands)
|   |   |   RuleEngine.evaluate(): First-match rule evaluation
|   |   |   Returns: (type, prop_id, is_common, depth, ry0, ry1)
|   |   ▼
DATABASE (data/cadastre_3d.db - SQLite)
|
|   7 Tables with FK constraints + indexes
|   PRAGMA foreign_keys = ON
|   Row factory: sqlite3.Row (dict-like access)
|
|   Data returned to services → routers → FastAPI → JSON serialization → HTTP response
|
|   ▼
USER RECEIVES: JSON response in browser / API client
|
|   For 3D visualization:
|   app.js receives mesh-data JSON → constructs Plotly Mesh3d traces
|   → WebGL renders in canvas → user interacts (rotate, zoom, hover, filter)
|
|   ▼
RENDERED 3D DIGITAL TWIN in browser

```

SECTION 35 — FINAL PROJECT SUMMARY

THE PROBLEM

India's land administration system cannot identify, validate, or administer property rights in multi-storey buildings. The entire DILRMP/ULPIN framework operates in 2D, treating a 10-storey building with 100 apartments as a single surface parcel. This creates unresolvable boundary disputes, mortgage fraud, unverifiable carpet areas, and administrative paralysis.

THE SOLUTION

The **National 3D ULPIN & Volumetric Cadastre Platform** — a working software system that transforms 2D floor plans into georeferenced, topologically validated, rights-linked 3D volumetric parcels with standardized 3D spatial identifiers conforming to ISO 19152 LADM v2.

THE CORE INNOVATION

A **config-driven, dual-mode extraction pipeline** with a declarative `RuleEngine` that allows any building in India to be onboarded via a JSON configuration file — zero code changes. Combined with automated 3D topology validation, anti-fraud encumbrance blocking, and unsupervised ML height verification.

THE TECHNOLOGY

- **Extraction:** PyMuPDF (vector) + OpenCV + Tesseract OCR (scanned) → automated room boundary detection
- **Georeferencing:** Local CAD → metric → WGS84 coordinate transformation
- **3D ULPIN:** `BUILDING-FLOOR-UNIT-TYPE` standardized composite identifier
- **Topology:** Shapely 2.0 computational geometry → 3D collision detection
- **Rights Registry:** ISO 19152 LADM v2 strata titles with capacity enforcement, common property locks, and conveyance blocking
- **ML:** DBSCAN + scipy peak detection for drone photogrammetry height validation
- **API:** FastAPI REST with 8 routers, Pydantic validation, auto-generated OpenAPI docs
- **Visualization:** Plotly WebGL 3D interactive digital twin with floor filtering and hover inspection
- **Database:** SQLite with 7 relational tables, foreign keys, and indexed spatial queries

THE USERS

Revenue officers, surveyors, property owners, banks, municipal corporations, RERA authorities, courts — every stakeholder in India's property ecosystem.

THE IMPACT

- **700 verified 3D volumetric parcels** generated from a real building's floor plan
- **100% topology compliance** — zero spatial collisions across 10 floors
- **Anti-fraud protections** — mortgage liens automatically block illegal resale
- **Automated extraction** — hours instead of weeks for building digitization
- **Standards compliance** — ISO 19152 LADM v2 alignment for international interoperability

THE SCALABILITY

The architecture is designed to scale from a single building (prototype) through city-scale (PostgreSQL + spatial indexing) to national deployment (microservices + federated databases). The config-driven design means onboarding 100,000 buildings requires 100,000 JSON files, not 100,000 code changes.

THE FUTURE

From a hackathon prototype to a national government service. The roadmap progresses through PostgreSQL migration, multi-building support, government API integration, deep learning extraction, and ultimately nationwide deployment as India's first true 3D cadastre — giving every room in every building a unique, immutable, government-recognized spatial identity.

END OF MASTER PROJECT DOCUMENT

Document prepared for: Smart India Hackathon 2026, PS 26011 Evaluation

Platform: National 3D ULPIN & Volumetric Cadastre Platform

Standard: ISO 19152 LADM v2 & Maharashtra Land Revenue (Survey 140/1)

Repository: github.com/adiscripts03/3D_UPLIN_SIH (https://github.com/adiscripts03/3D_UPLIN_SIH).

